Abstract Interpretation with Error Bounds

ANONYMOUS AUTHOR(S)

Abstract interpretation provides sound over-approximations for static program analysis, but soundness often comes with imprecision. Distance-based partial completeness quantifies this imprecision by measuring the distance between the precise abstract image and the result computed by an analyzer. Existing techniques mainly derive upper bounds on this imprecision, which certify how imprecise the analysis may be but do not reveal how much imprecision is unavoidable. This paper develops a two-sided framework that, given a program, an input set, and an abstract-domain distance, derives an interval $[\ell, u]$ enclosing the exact imprecision. The upper bound u is tightened by replacing generator-only propagation with representative concrete states that better preserve structure under later restrictions. The lower bound ℓ is propagated compositionally and can be improved using false-positive feedback as semantic constraints. We instantiate the framework in a two-dimensional numerical setting where convex hulls model abstract elements and area difference measures imprecision. Experiments show that representative-state selection substantially tightens upper bounds over generator-only propagation, while posterior information raises initially conservative lower bounds.

1 Introduction

Abstract interpretation [5, 7, 8] has served as a foundational theory for static program analysis. Since non-trivial semantic properties of programs are undecidable in general [16], practical analyzers inevitably approximate program semantics. The central idea of abstract interpretation is to compute an over-approximation of all reachable program states. Such an approximation guarantees *soundness*: a sound analysis does not miss any target defect of the kind it is designed to detect. This principle has enabled the deployment of static analysis tools in both academia and industry [10, 14, 17, 18]. However, soundness is usually obtained at the cost of *imprecision*, leading to spurious warnings and false positives in practical analyzers. This imprecision has long limited the practical adoption of static analysis tools. It is therefore important not only to prove that an analysis is sound, but also to quantify the precision loss introduced by abstraction.

A classical way to characterize the absence of precision loss is *completeness*. Given a concrete semantics $f^C : C \to C$, an abstraction function $\alpha : C \to \mathcal{A}$, and an abstract transformer $f^{\mathcal{A}} : \mathcal{A} \to \mathcal{A}$, completeness requires $\alpha(f^C(c)) = f^{\mathcal{A}}(\alpha(c))$. Global completeness requires this equality to hold for all input sets, while local completeness restricts the requirement to a particular input set. Both notions, however, remain too strict for practical analyzers: they distinguish exact preservation from any non-zero loss, but do not quantify whether the loss is negligible or severe. To reason about practical precision, we need a quantitative relaxation of completeness.

A prominent line of work provides such a relaxation by measuring precision loss through an abstract-domain distance: $\delta_{\mathcal{A}} : \mathcal{A} \times \mathcal{A} \to \mathbb{R}^{\infty}_{\geq 0}$ measured over the elements of the abstract domain. Campion et al. [3] introduce *ε-partial local completeness*: Instead of asking whether $\alpha(f^C(c)) = f^{\mathcal{A}}(\alpha(c))$, partial completeness asks whether their abstract-domain distance $\delta_{\mathcal{A}}(\alpha(f^C(c)), f^{\mathcal{A}}(\alpha(c)))$ is bounded by a given threshold ε. In other words, it transforms the classical binary notion of completeness into a quantitative measure of how much the abstract interpretation may over-approximate the concrete behavior. While partial completeness gives a quantitative specification of acceptable precision loss, practical analyzers require a way to reason about how such loss accumulates across program constructs. This motivated the extended *Error Propagation Logic* (EPL) [4] to inductively compute *upper bounds* on the introduced imprecision.

However, this line of work also leaves two limitations unresolved. First, it derives only *upper bounds* on the imprecision. An upper bound tells us how bad the abstraction may become, but it does not tell us how much imprecision is already unavoidable. Consequently, a small upper bound certifies precision, but a large upper bound is ambiguous: it may indicate that the analysis

is genuinely imprecise, or merely that the bound is conservative. Without a corresponding lower bound, we cannot distinguish unavoidable precision loss from artifacts of the estimation method.

Second, EPL relies on *generators*, i.e., minimal concrete representatives of abstract elements, to reduce the search space for upper-bound reasoning. This is effective for preserving the current abstract property, but it is not necessarily stable under later program constructs. After state-filtering operations such as Boolean guards, many generator states may be discarded, and the surviving states may no longer contain enough concrete information to represent the filtered property accurately. As a result, generator-only reasoning can propagate unnecessarily loose upper bounds, especially when future restrictions expose parts of the concrete state space that were not represented by the original generators.

Our paper advances the previous work in two directions. First, we generalize one-sided error propagation into a *two-sided* quantitative account, deriving both *upper* and *lower* bounds of abstract imprecision simultaneously. Given a concrete input set I, a program P, and an abstract-domain distance $\delta_{\mathcal{A}}$, we reason about the true imprecision $\delta_{\mathcal{A}}\,(\alpha(\llbracket \mathsf{P} \rrbracket_C(I)), \llbracket \mathsf{P} \rrbracket_{\mathcal{A}}(\alpha(I)))$ through an upper bound u and a lower bound l. Second, our framework also improves the tightness of upper-bound reasoning. Instead of relying only on generators, we allow additional representative concrete states to be retained. Generators guarantee that the current abstraction is represented exactly, while extra representative states preserve richer concrete structure that may become relevant after future filtering or merging operations. This yields tighter sound upper bounds in situations where generator-only reasoning is overly conservative.

The two-sided view also gives posterior information a natural role. When external evidence is available, such as a user confirming that an alarm is spurious, it can be interpreted as an additional semantic constraint on the possible imprecision. This lets the framework tighten conservative bounds without changing the underlying abstract interpretation result.

We instantiate the framework on a two-dimensional numerical setting, where convex hulls model abstract elements and area difference measures imprecision. The experiments evaluate three parts of the framework: representative-state selection for upper bounds, forward propagation for initial lower bounds, and posterior learning from false-positive alarm triage. Across 60,000 input-set–program pairs, our representative-based upper-bound estimates improve over the generator baseline by more than 91%, showing that the resulting upper bounds are much tighter. The initial lower bound is intentionally conservative, but still recovers 46.3% of the loss at the p_t program point on average; with posterior feedback, this recovery rises to 64.4%. These results show that posterior learning is not a prerequisite for the framework, but it provides a useful additional advantage when alarm-triage feedback is available.

In summary, this paper makes the following contributions:

- We extend the distance-based view of partial completeness with a compositional two-sided propagation methodology, deriving both lower and upper bounds $[\ell, u]$ on abstract-domain distances. This allows us to reason not only about how large the imprecision may become, but also about how much imprecision must already be present.
- We refine generator-based error propagation by augmenting generators with additional representative concrete states. This preserves more information for subsequent state-filtering operations and can derive tighter upper bounds than generator-only reasoning.
- We show that two-sided imprecision bounds can be updated using posterior information. Evidence such as user annotations and dynamic test results can be used to tighten and propagate the lower and upper bounds of abstract imprecision.
- We instantiate our framework on a two-dimensional numerical setting and empirically evaluate both sides of the framework, showing that representative coverage substantially tightens upper bounds and that posterior feedback can improve conservative lower bounds.

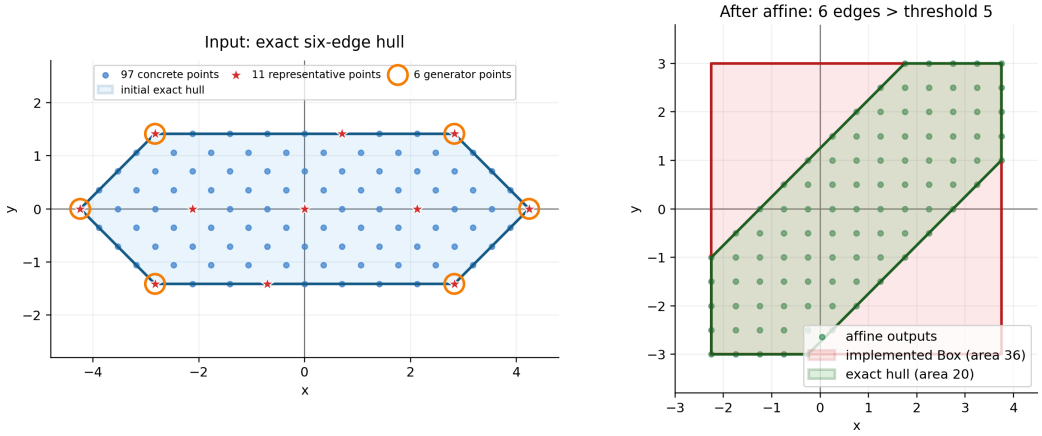


Fig. 1. The input set of two-dimensional points and the abstractions. Red stars in the left panel mark the 11 selected points used for tighter upper-bound propogation in Section 2.4; orange hollow rings identify the six generator points among them. The initial six-edge hull is kept exactly. After the affine transfer, its six edges exceed the threshold 5, so the analyzer over-approximates the area-20 hull by an area-36 axis-aligned box.

2 Overview

This section illustrates our two-sided framework through a two-dimensional example. We first execute a conventional abstract interpreter and expose the gap between its abstract results and the exact reachable sets. Starting from the EPL upper bound [4], we show how representative states tighten this upper bound. Our framework also computes a lower bound on the same gap, and posterior information can further raise this lower bound. The presentation is intentionally informal; the general definitions and soundness results appear in the following sections.

2.1 A Running Numerical Analysis and its Exact Imprecision

Consider program states as points $\boldsymbol{v} = (x, y) \in \mathbb{R}^2$. The input set I contains 97 points whose convex hull is a horizontally elongated hexagon as shown in the left panel of Figure 1. Our analyzer uses the convex-polyhedra domain Poly, but maintains a simple complexity budget: after each transfer function, a polygon with more than five edges will be over-approximated by its smallest enclosing axis-aligned box. This reflects a common engineering tradeoff—polyhedra preserves relational information, whereas boxes are cheaper to store and propagate.

We use the following example program throughout this section:

$\boldsymbol{v}$:= affine($\boldsymbol{v}$); **if** $x \geq 0$ **then** assert($y - x \geq -3$) **else** $x := x - \frac{1}{2}$; assert($y \leq \frac{5}{2}$);

Here, affine($\boldsymbol{v}$) first rotates every state point counterclockwise by 45° around the origin and then translates it by 3/4 in the positive x direction. Equivalently, it is the assignment $\boldsymbol{v} := A\boldsymbol{v} + b$ with $A = \begin{pmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{pmatrix}$ and $b = \begin{pmatrix} 3/4 \\ 0 \end{pmatrix}$. This affine transformation preserves area. We call the assertion in the then branch $\texttt{assert}_1$ and the one after the branch join $\texttt{assert}_2$. The else branch contains an assignment statement, and hence the two branches exhibit different behaviors.

As shown in the right panel of Figure 1, the initial polygon is not abstracted to an axis-aligned box immediately. After the affine transfer, its exact hull is $H = \text{Conv}\{(-2.25, -1), (1.75, 3), (3.75, 3), (3.75, 1), (-0.25, -3), (-2.25, -3)\}$. It still has six edges and area 20, and is therefore abstracted

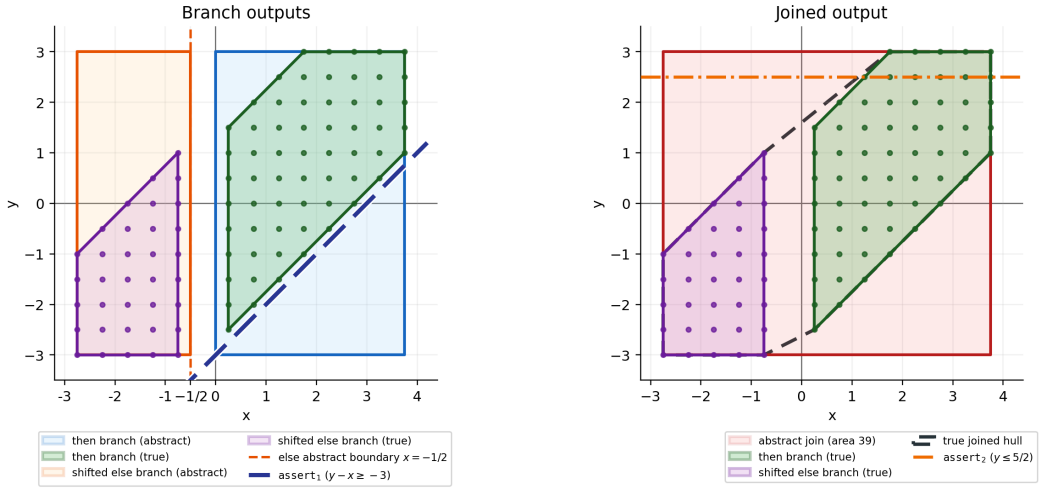


Fig. 2. Abstract and exact results after the two guards and after the branch join. The dots show the reachable concrete points. In the right panel, green and purple retain the two exact branch results, and the dashed outline marks their exact convex join. The left panel marks $\texttt{assert}_1$, while the right marks $\texttt{assert}_2$.

at that point by the axis-aligned box B. Here, $B = [-2.25, 3.75] \times [-3, 3]$, whose area is 36. For this example, we measure imprecision by area: $err(A, T) \triangleq \text{Area}(A) - \text{Area}(T)$, where A is the analyzer's produced polygon and T is the convex hull of the reachable concrete points. Thus, the Box conversion introduces error $36 - 20 = 16$.

We next run the abstract interpreter through the two branches as shown in Figure 2. The guard $x \geq 0$ restricts B to an area-22.5 rectangle, while the hull of the reachable states is an area-12 pentagon. Hence, at $\texttt{assert}_1$, the exact imprecision is $22.5 - 12 = 10.5$. The else guard produces an area-13.5 abstract rectangle. Because the guard is strict, after the assignment $x := x - \frac{1}{2}$ the exact hull has right boundary $x = -1$ and area 6. Joining the branch results gives an area-39 abstract rectangle, whereas the hull of the concrete joined output has area 22.375. Consequently, the exact imprecision at $\texttt{assert}_2$ is $39 - 22.375 = 16.625$. These exact values are ground truth used to explain the example. The abstract result at the then branch violates $y - x \geq -3$, although every reachable state satisfies it, so $\texttt{assert}_1$ is a false positive. In contrast, reachable joined states with $y = 3$ violate $y \leq \frac{5}{2}$, making $\texttt{assert}_2$ a real error.

2.2 EPL Upper Bounds

The EPL [4] derives an upper bound on abstract imprecision by propagating generators. It propagates only the six vertices of the input hull H in Figure 1. These vertices form the support set of H: they are the extreme input points whose convex hull is H. As shown in the middle panel of Figure 3, after the then guard $x \geq 0$, the surviving support points have a hull of area 2. Pure generator propagation therefore gives the upper bound $22.5 - 2 = 20.5$ at $\texttt{assert}_1$. Propagating the same support points through both branches yields a joined hull of area 21, and hence the upper bound $39 - 21 = 18$ at $\texttt{assert}_2$. Moreover, EPL has supplied the upper endpoints but no lower endpoint.

2.3 Initial Lower Bounds

Our framework complements this estimation of imprecision with a lower bound. Before the branch, the known loss is 16, which is captured precisely by our framework. A guard may remove spurious

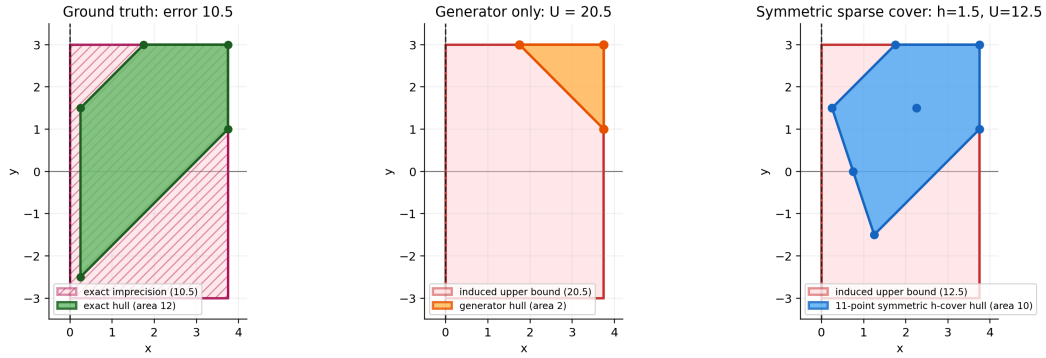


Fig. 3. Upper bounds at $\texttt{assert}_1$. The generator hull retains area 2, whereas the sparse h-cover with $h = 1.5$ retains area 10 of the exact area 12. Hatched light red denotes the exact imprecision; unhatched light red denotes the induced upper bound.

area together with the abstract region it discards; in the worst case, the entire removed region may be imprecision already accounted for the certified loss. For the then branch, the guard removes the area-13.5 part of B with $x < 0$, so the lower bound at $\texttt{assert}_1$ becomes $\max(16 - 13.5, 0) = 2.5$. The else branch removes area 22.5 and therefore carries lower bound 0. When the branches join, the forward propagation does not remember the shape of the loss certified in the then branch. In the worst case, this area-2.5 loss may lie near the y-axis and be filled by the other branch at the join, so the initial lower bound at $\texttt{assert}_2$ is still 0. Combining the EPL upper bound with our initial lower bound gives the certified interval $[2.5, 20.5]$ on the exact imprecision 10.5 of $\texttt{assert}_1$ and $[0, 18]$ on the imprecision 16.625 of $\texttt{assert}_2$.

2.4 Tighter Upper Bounds with Representative States

Generators describe the boundary of the input hull, but they cannot represent its interior after a later restriction. Our upper-bound method therefore retains a configurable set of representative states. In this example, we construct a sparse, centrally symmetric h-cover of the 97 input points. An h-cover means that every input point lies within Euclidean distance h of some selected representative; a smaller h corresponds to finer Euclidean coverage. Our cover contains the six generators, the origin, and two pairs of opposite interior points, for a total of 11 representatives as shown in Figure 1. Its achieved covering radius is $h = \max_{p \in I} \min_{r \in R} \|p - r\|_2 = 1.5$. This coverage matters because more representatives may survive a later restriction. As Section 6 formalizes, these surviving representatives can preserve the restricted concrete set more faithfully, yielding a probabilistic bound on the resulting abstract loss. The general representative-selection method and its theoretic guarantees are given in Section 6.

Figure 3 shows the effect at $\texttt{assert}_1$. The original input set contains 97 points, while the generator baseline stores six and our method stores only 11—just five additional representatives as in Figure 1. This modest storage cost raises the surviving hull area to 10 and improves the upper bound from $22.5 - 2 = 20.5$ to $22.5 - 10 = 12.5$; relative to the true error 10.5, the excess in the upper bound drops from 10 to 2. At $\texttt{assert}_2$, the same representatives yield a joined hull of area 21.625 and the upper bound $39 - 21.625 = 17.375$, already close to the true error 16.625. Thus, retaining only a small fraction of the input states substantially tightens the upper-bound estimates.

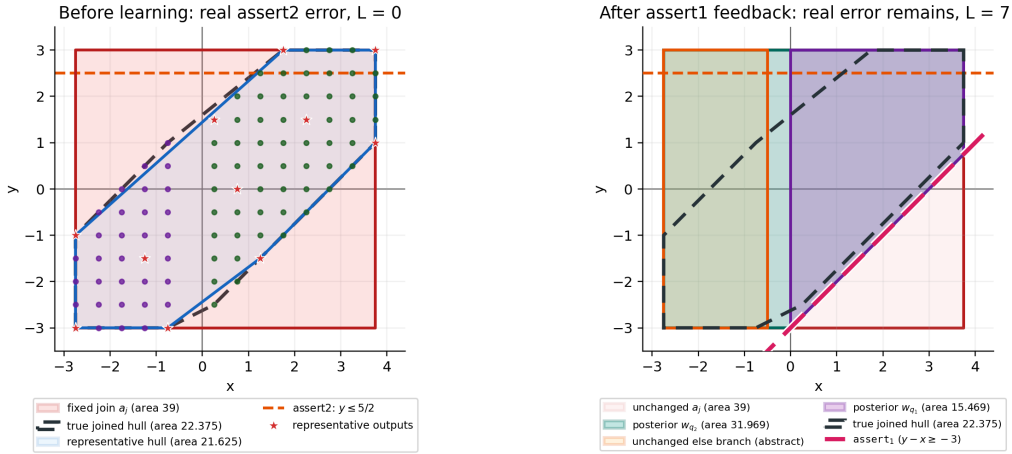


Fig. 4. Posterior information from $\texttt{assert}_1$ propagates through the join and raises the lower bound at $\texttt{assert}_2$ from 0 to 7.031. The left panel shows the concrete outputs together with the representatives and their hull; the right panel focuses on w_{q_1} and w_{q_2}.

2.5 Raising Lower Bounds through Posterior Learning

The forward propagation in Section 2.3 already provides an initial lower endpoint, even when it is conservative. Posterior feedback is an additional advantage: once an alarm is confirmed to be a false positive, the feedback can be turned into a posterior restriction and propagated to related sites. Such feedback is typical in alarm triage, where users inspect selected reports and mark some of them as false positives. We illustrate this process using feedback from $\texttt{assert}_1$.

The feedback establishes that all concrete states at that site satisfy the asserted region $R_1 = \{(x, y) \mid y - x \ge -3\}$. Therefore, we maintain a posterior abstract result $w_{q_1} = a_t \cap R_1$. Notably, w_{q_1} has five edges, so it remains within the complexity threshold and can be retained exactly. More importantly, the learned information benefits the later, unchecked alarm at $\texttt{assert}_2$. Since the else branch receives no feedback, its posterior result remains its original area-13.5 abstract rectangle. Joining it with w_{q_1} produces a five-edge posterior polygon w_{q_2} of area 31.969 (still within the threshold and retained exactly). We do not directly modify the area-39 abstract join; instead, comparing it with the posterior result updates the lower bound at $\texttt{assert}_2$ from 0 to $\mathrm{Area}(a_j) - \mathrm{Area}(w_{q_2}) = 39 - 31.969 = 7.031$. This tighter lower bound certifies more unavoidable imprecision at the unreviewed site, and thus increases the degree of suspicion attached to its alarm.

The feedback does not remove the real alarm at $\texttt{assert}_2$: concrete joined states still violate its specification. It improves only our quantitative account of the analyzer's imprecision. Combining the representative-state upper bounds with posterior learning narrows the certificate at $\texttt{assert}_2$ to $[0, 18] \longrightarrow [7.031, 17.375]$.

To conclude, this overview example previews the two complementary parts of our framework: representative states tighten upper-bound estimates, while our lower-bound calculation supplies the missing lower endpoint and posterior information can raise it further.

3 Preliminaries

We fix the notation and semantic conventions used in the rest of the paper. The presentation separates the concrete execution model from its abstract counterpart and then recalls the precision notions on which our two-sided bounds are based.

$$\llbracket \texttt{skip} \rrbracket_C\, c \triangleq c \qquad \llbracket v := a \rrbracket_C\, c \triangleq \{\sigma[v \mapsto (\!|a|\!)\,\sigma] \mid \sigma \in c\} \qquad \llbracket b? \rrbracket_C\, c \triangleq \{\sigma \in c \mid (\!|b|\!)\,\sigma = \text{True}\}$$
$$\llbracket s_1; s_2 \rrbracket_C\, c \triangleq \llbracket s_2 \rrbracket_C\, (\llbracket s_1 \rrbracket_C\, c) \qquad \llbracket s_1 \oplus s_2 \rrbracket_C\, c \triangleq \llbracket s_1 \rrbracket_C\, c \cup \llbracket s_2 \rrbracket_C\, c \qquad \llbracket s^* \rrbracket_C\, c \triangleq \bigcup_{n \in \mathbb{N}} (\llbracket s \rrbracket_C)^n c$$

Fig. 5. Collecting concrete semantics $\llbracket \cdot \rrbracket_C$.

3.1 Basic Notations

Lattices. We write $L = \langle L, \leq_L, \vee_L, \wedge_L, \top_L, \bot_L \rangle$ for a complete lattice, with partial order $\leq_L$, lub (least upper bound) $\vee_L$, glb (greatest lower bound) $\wedge_L$, greatest element $\top_L$ and least element $\bot_L$. Subscripts are omitted when the underlying lattice is unambiguous. For a set X, its powerset complete lattice $\mathcal{P}(X)$ is ordered by inclusion ($\subseteq$); its $\vee$ and $\wedge$ are union ($\cup$) and intersection ($\cap$).

Functions. A function $f : X \to Y$ also denotes its pointwise image map on sets, $f(S) \triangleq \{f(x) \mid x \in S\}$. Function composition is written as $g \circ f$, with f applied first. For an endofunction $f : X \to X$, let $f^0 \triangleq id_X$ (the identity function on set X) and $f^{n+1} \triangleq f^n \circ f$. A function between complete lattices is *monotone* when it preserves their orders; a function $f : L_1 \to L_2$ is *additive* when for all $x, y \in L_1, f(x \vee_{L_1} y) = f(x) \vee_{L_2} f(y)$. We compare lattice-valued functions pointwise: $f \leq_L g$ means $f(x) \leq_L g(x)$ for every $x \in L$, and we denote: $(f \vee_L g)(x) \triangleq \lambda x. f(x) \vee_L g(x)$. When the least fixpoint of $f : L \to L$ exists, it is denoted by $\mathsf{lfp}\,(f)$.

3.2 Programs

We use a small language of standard imperative programs as the common control-flow skeleton for the concrete and abstract semantics:

$$\textsc{Program} \qquad \mathit{Prog} \ni s ::= e \in \mathit{BaseCmd} \mid s; s \mid s \oplus s \mid s^*.$$

Here $s; s$ is sequencing, $s \oplus s$ is nondeterministic choice, and s^* represents the Kleene iteration. This core language is standard in semantic accounts of imperative programs and Kleene-algebraic models [2, 3, 12, 15, 23, 24]. For the general development, the set *BaseCmd* of basic commands is drawn from a conventional real-valued while-language:

$$\begin{aligned}
&\textsc{Arithmetic Expression} && \mathit{AExp} \ni a ::= v \in \mathit{Var} \mid n \in \mathbb{R} \mid a + a \mid a - a \mid a \times a,\\
&\textsc{Boolean Expression} && \mathit{BExp} \ni b ::= \text{True} \mid \text{False} \mid a = a \mid a < a \mid a \leq a \mid \neg b \mid b \wedge b,\\
&\textsc{Basic Command} && \mathit{BaseCmd} \ni e ::= \texttt{skip} \mid v := a \mid b?.
\end{aligned}$$

It includes no-op instructions, assignments, Boolean guards, and real-valued arithmetic expressions. Moreover, the if-then-else branching and while-loop commands presented in the standard deterministic imperative language [23] can be defined as syntactic sugar:

$$\texttt{if}\ b\ \texttt{then}\ s_1\ \texttt{else}\ s_2 \triangleq (b?; s_1) \oplus (\neg b?; s_2) \qquad \texttt{while}\ b\ \texttt{do}\ s \triangleq (b?; s)^*; \neg b?.$$

3.3 Concrete Semantics of Programs

In this paper, we fix a finite variable set *Var*. A program state $\sigma : \mathit{Var} \to \mathbb{R}$ assigns a real value to each variable, and the state space is $\Sigma \triangleq \mathit{Var} \to \mathbb{R}$. For $\sigma \in \Sigma$, the update $\sigma[v \mapsto n]$ agrees with σ except that it maps v to n. Arithmetic and Boolean expressions are interpreted by the usual evaluation functions, both written $(\!|\cdot|\!)$ when their type ($\mathit{AExp} \to \Sigma \to \mathbb{R}$ or $\mathit{BExp} \to \Sigma \to \{\text{True}, \text{False}\}$) determines the meaning.

The collecting semantics maps a set of input states to all states reachable from it. Its defining equations are collected in Figure 5. This definition is standard for designing static program analysis [7]. Thus $\llbracket \cdot \rrbracket_C : \mathit{Prog} \to \mathcal{P}(\Sigma) \to \mathcal{P}(\Sigma)$ is determined compositionally from the basic commands.

3.4 Abstract Interpretation

We recall only the abstract-interpretation concepts needed to state our precision bounds [6, 8, 9]. Let the concrete domain C and the abstract domain $\mathcal{A}$ be complete lattices linked by a Galois connection $\langle \alpha : C \to \mathcal{A}, \gamma : \mathcal{A} \to C \rangle$, where α and γ are monotone functions and $\alpha(c) \leq_{\mathcal{A}} a \Leftrightarrow c \leq_C \gamma(a)$. holds. Unless stated otherwise, C is the collecting domain $\mathcal{P}(\Sigma)$ introduced above. The order on $\mathcal{A}$ is a precision order: $a \leq_{\mathcal{A}} b$ says that b is at least as coarse an approximation as a.

Sound abstract transfers. Consider a monotone concrete transfer function $f : C \to C$ and an abstract transfer function $f^{\mathcal{A}} : \mathcal{A} \to \mathcal{A}$ (both are simplified as transfers in the following text). Soundness is the pointwise inequality $\alpha \circ f \leq_{\mathcal{A}} f^{\mathcal{A}} \circ \alpha$. Consequently, $f(c) \leq_C \gamma(f^{\mathcal{A}}(\alpha(c)))$ for every input set c. Among sound transfers, the *best correct approximation* (bca) of f on $\mathcal{A}$ is $f^{\alpha} \triangleq \alpha \circ f \circ \gamma$. Every sound $f^{\mathcal{A}}$ satisfies: $f^{\alpha} \leq_{\mathcal{A}} f^{\mathcal{A}}$. The standard fixpoint soundness result likewise gives $\alpha(\mathsf{lfp}\,(f)) \leq_{\mathcal{A}} \mathsf{lfp}\,(f^{\mathcal{A}})$ whenever the relevant fixpoints exist.

From completeness to partial completeness. A sound transfer is globally *complete* when $\alpha \circ f = f^{\mathcal{A}} \circ \alpha$ holds. This intuitively asks for exact abstract behavior at every input set. Local completeness [2] restricts this optimal precision to a collection of input sets $S \subseteq C$: $\forall c \in S, \alpha(f(c)) = f^{\mathcal{A}}(\alpha(c))$. Taking $S = C$ recovers global completeness, whereas $S = \{c\}$ isolates a single input set.

Partial completeness replaces the equality test with a quantitative allowance [3, 4]. Let $\delta_{\mathcal{A}} : \mathcal{A} \times \mathcal{A} \to \mathbb{R}^{\infty}_{\geq 0}$ measure the imprecision incurred when one abstract element approximates another. For $\varepsilon \in \mathbb{R}^{\infty}_{\geq 0}$, $f^{\mathcal{A}}$ is *ε-partially locally complete for f on c* if $\delta_{\mathcal{A}}\big(\alpha(f(c)), f^{\mathcal{A}}(\alpha(c))\big) \leq \varepsilon$. The definition extends pointwise to $S \subseteq C$. Because soundness orders $\alpha(f(c)) \leq_{\mathcal{A}} f^{\mathcal{A}}(\alpha(c))$, the measured gap is the extra region or behavior admitted by the abstract computation. With a distance that separates comparable elements, 0-partially locally complete is precisely local completeness.

The choice of $\delta_{\mathcal{A}}$ determines what the budget ε means and which analyses count as partially complete. Following the order-compatible distance structures used for partial completeness [3, 4], we use a *strong order-compatible pre-metric*, and it satisifies the following axioms for all $x, y, z \in \mathcal{A}$:

$$
\begin{aligned}
&\textit{(if-identity)} && x = y \Rightarrow \delta_{\mathcal{A}}(x, y) = 0 \\
&\textit{(chain-order)} && x \leq_{\mathcal{A}} y \leq_{\mathcal{A}} z \Rightarrow \delta_{\mathcal{A}}(x, y) \leq \delta_{\mathcal{A}}(x, z) \wedge \delta_{\mathcal{A}}(y, z) \leq \delta_{\mathcal{A}}(x, z) \\
&\textit{(chain iff-identity)} && x \leq_{\mathcal{A}} y \Rightarrow (\delta_{\mathcal{A}}(x, y) = 0 \Rightarrow x = y) \\
&\textit{(triangle-inequality)} && \delta_{\mathcal{A}}(x, z) \leq \delta_{\mathcal{A}}(x, y) + \delta_{\mathcal{A}}(y, z).
\end{aligned}
$$

The first and third axioms make zero error coincide with equality along the precision order, which yields the correspondence between 0-partial and ordinary local completeness. The chain-order axiom aligns numerical loss with decreasing abstract precision, while the triangle inequality permits bounds from successive approximation steps to be composed.

Example (volume distance). For a numerical domain $\mathcal{N}$ such as box, zone, or octagon, let the volume function $Vol : \mathcal{N} \to \mathbb{R}^{\infty}_{\geq 0}$ be monotone with respect to concretization inclusion. Then $\delta^{Vol}_{\mathcal{N}}(N_1, N_2) \triangleq Av\big(Vol(N_2) - Vol(N_1)\big)$ measures numerical over-approximation, where Av denotes the absolute value. It is a strong order-compatible pre-metric that can distinguish comparable domain elements, i.e., $N_1 \leq_{\mathcal{N}} N_2$ and $Vol(N_1) = Vol(N_2)$ imply $N_1 = N_2$. This is the instance used by our two-dimensional examples.

3.5 Abstract Semantics

We inherit the abstract semantic inferface used for local and partial completeness [2, 3], where an analysis supplies a sound transfer $[\![e]\!]_{\mathcal{A}} : \mathcal{A} \to \mathcal{A}$ for every basic command. These transfers

induce a denotation $[\![\cdot]\!]_{\mathcal{A}} : \mathit{Prog} \to \mathcal{A} \to \mathcal{A}$ by structural recursion:

$$\begin{aligned} [\![s_1; s_2]\!]_{\mathcal{A}} &\triangleq [\![s_2]\!]_{\mathcal{A}} \circ [\![s_1]\!]_{\mathcal{A}}, \\ [\![s_1 \oplus s_2]\!]_{\mathcal{A}}(a) &\triangleq [\![s_1]\!]_{\mathcal{A}}(a) \vee_{\mathcal{A}} [\![s_2]\!]_{\mathcal{A}}(a), \\ [\![s^*]\!]_{\mathcal{A}}(a) &\triangleq \mathsf{lfp}\,(\lambda x.\ a \vee_{\mathcal{A}} [\![s]\!]_{\mathcal{A}}(x))\,. \end{aligned}$$

For each basic command e, soundness requires $\alpha \circ [\![e]\!]_C \leq_{\mathcal{A}} [\![e]\!]_{\mathcal{A}} \circ \alpha(c)$. This compositional definition deliberately leaves the abstract domain and its transfer implementation open, and makes it general enough to cover the analyses considered later. We also assume that the chosen analyzer terminates in a finite number of steps on all analyzed input sets; finite-height domains, widening, or other analysis-specific convergence mechanism can ensure this assumption.

4 Two-Sided Partial Completeness Framework

Partial completeness weakens the classical notion of completeness by permitting an explicitly bounded amount of imprecision [3], while the existing error-propagation logic (EPL) [4] derives an *upper bound* on this loss. Such a bound answers an important certification question: how much imprecision can the abstract execution introduce in the worst case? If the upper bound is at most a tolerance ε, then ε-partial completeness is established. However, a large upper bound is inconclusive. It may reflect unavoidable imprecision in the analysis, or it may merely be a loose estimate caused by conservative propagation. An upper bound alone cannot distinguish these two cases.

We therefore extend partial completeness with a complementary lower bound. The upper bound limits how imprecise the abstract execution *may be*, whereas the lower bound certifies how much imprecision *must already be present*. Together, they locate the true analysis-induced imprecision inside an interval.

4.1 Exact Local Imprecision

Let C and $\mathcal{A}$ be concrete and abstract domains connected by a Galois connection $\langle \alpha, \gamma \rangle$. Let $f : C \to C$ be a concrete semantic function and let $f^{\mathcal{A}} : \mathcal{A} \to \mathcal{A}$ be a sound abstract approximation of $f : C \to C$. Soundness gives $\alpha(f(I)) \leq_{\mathcal{A}} f^{\mathcal{A}}(\alpha(I))$ for every concrete input set $I \in C$. Thus, the two abstract elements are comparable, and their distance measures the additional over-approximation introduced by the abstract computation.

Definition 4.1 (Exact local imprecision)**.** Let $\delta_{\mathcal{A}} : \mathcal{A} \times \mathcal{A} \to \mathbb{R}^{\infty}_{\geq 0}$ be a strong order-compatible pre-metric. The *exact local imprecision* of $f^{\mathcal{A}}$ with respect to f on $I \in C$ is

$$\mathsf{err}_{\mathcal{A}}(f, f^{\mathcal{A}}, I) \triangleq \delta_{\mathcal{A}}\big(\alpha(f(I)), f^{\mathcal{A}}(\alpha(I))\big).$$

This quantity is the error used by ordinary partial completeness: $f^{\mathcal{A}}$ is ε-partially locally complete for f on I exactly when $\mathsf{err}_{\mathcal{A}}(f, f^{\mathcal{A}}, I) \leq \varepsilon$. In particular, when $\delta_{\mathcal{A}}$ distinguishes comparable elements at distance zero, $\mathsf{err}_{\mathcal{A}}(f, f^{\mathcal{A}}, I) = 0$ is equivalent to local completeness on I.

For a program P and a concrete input set I, we instantiate the definition with its concrete and abstract semantics: $\mathsf{err}_{\mathcal{A}}(\mathsf{P}, I) \triangleq \delta_{\mathcal{A}}\left(\alpha([\![\mathsf{P}]\!]_C(I)), [\![\mathsf{P}]\!]_{\mathcal{A}}(\alpha(I))\right)$. This is the semantic quantity that our framework seeks to bound. Computing it directly may require the exact collecting semantics of P, and is therefore generally unavailable to a static analyzer.

4.2 Two-Sided Partial Completeness

Our extension replaces an exact but typically unavailable error with a sound interval enclosing it.

Definition 4.2 (Two-sided partial-completeness certificate)**.** Let $f^{\mathcal{A}} : \mathcal{A} \to \mathcal{A}$ be a sound approximation of $f : C \to C$, and let $I \in C$. A pair $[\ell, u] \in \mathbb{R}^{\infty}_{\geq 0} \times \mathbb{R}^{\infty}_{\geq 0}$ is a *two-sided partial-completeness certificate* for $f^{\mathcal{A}}$ with respect to f on I if $0 \leq \ell \leq \mathrm{err}_{\mathcal{A}}(f, f^{\mathcal{A}}, I) \leq u$. We call ℓ the *certified lower imprecision bound* and u the *certified upper imprecision bound.*

The certificate extends the one-sided account without changing the underlying notion of partial completeness. A upper-bound result u in previous work [3, 4] is the special certificate $[0, u]$. An exact computation yields $[\mathrm{err}_{\mathcal{A}}(f, f^{\mathcal{A}}, I), \mathrm{err}_{\mathcal{A}}(f, f^{\mathcal{A}}, I)]$. The interval $[0, 0]$ certifies local completeness. The width $u - \ell$ measures uncertainty in the certificate, not imprecision of the abstract interpretation itself. Consequently, improving the bounding procedure means increasing ℓ, decreasing u, or both, while preserving the enclosure of the same exact local imprecision.

4.3 The Two-Sided Imprecision Framework

The above definitions state the semantic contract of our paper independently of a particular abstract domain. They lead to the following general framework.

Definition 4.3 (Two-sided imprecision framework)**.** Given

- a concrete input set $I \in C = \mathcal{P}(\Sigma)$ and a program P;
- a Galois connection $\langle \alpha, \gamma \rangle$ along with a strong order-compatible pre-metric $\delta_{\mathcal{A}}$ over $\mathcal{A}$;
- concrete and sound abstract semantics $[\![\mathsf{P}]\!]_C$ and $[\![\mathsf{P}]\!]_{\mathcal{A}}$,

our approach derives quantities $L(\mathsf{P}, I)$ and $U(\mathsf{P}, I)$ such that $L(\mathsf{P}, I) \leq \mathrm{err}_{\mathcal{A}}(\mathsf{P}, I) \leq U(\mathsf{P}, I)$.

This is the main framework of our paper. The following two sections instantiate this framework from the lower-bound and upper-bound sides respectively: Section 5 develops a compositional lower-bound logic, and section 6 develops a representative-state method to tighten upper bounds.

5 Lower-Bound Reasoning

We first develop the lower-bound side of the two-sided certificate. The goal is to certify how much imprecision must already be present in an abstract execution. Our framework has a core layer and an optional refinement layer. The core layer is a logic that propagates lower bounds through program structure. The optional layer uses posterior constraints, when available, to refine initially conservative lower bounds after observing that some alarms are spurious. Finally, we instantiate the lower-bound rules in the two-dimensional setting used in our experiments. We start with the forward logic, following the proof-system style of prior logics for imprecision propagation [3, 4].

5.1 Forward Propagation Logic

Recall that the exact imprecision of a program P on a fixed concrete input set $I \in C$ is $\mathrm{err}_{\mathcal{A}}(\mathsf{P}, I) \triangleq \delta_{\mathcal{A}}\left(\alpha([\![\mathsf{P}]\!]_C(I)), [\![\mathsf{P}]\!]_{\mathcal{A}}(\alpha(I))\right)$. We denote a lower-bound predicate $\ell - \textsc{LBound}(\mathsf{P}, I)_{\mathcal{A}}$ which means that $\ell \in \mathbb{R}^{\infty}_{\geq 0}$ is a lower bound on this exact imprecision: $\ell \leq \mathrm{err}_{\mathcal{A}}(\mathsf{P}, I)$. This predicate is the external specification of the forward analysis on the fixed input set I. Internally, the analysis maintains at each program point a pair (a, l), where $a \in \mathcal{A}$ is the current abstract result and $l \in \mathbb{R}^{\infty}_{\geq 0}$ is the currently certified lower bound. We write $\langle a, l \rangle \overset{\mathsf{P}}{\Rightarrow}_{L,\mathcal{A}} \langle a', l' \rangle$ when executing P soundly transforms the input pair (a, l) into the output pair (a', l'). Consequently, intermediate derivations need not retain the original concrete input set; it appears only when the initial pair is formed and the final lower-bound predicate is concluded.

A key auxiliary construction is the set of possible lower-bound witnesses. If the abstract analysis produces $a \in \mathcal{A}$ and we already know that the exact abstract image is at distance at least ε from a, then the exact image must lie in $\mathrm{Wit}_L(a, \varepsilon) \triangleq \{a' \in \mathcal{A} \mid a' \sqsubseteq_{\mathcal{A}} a \text{ and } \delta_{\mathcal{A}}(a', a) \geq \varepsilon\}$. We use Wit_L

$$\frac{\textsc{LTrans}_{\mathcal{A}}(e,a,l,a',l')}{\langle a,l\rangle \overset{e}{\Longrightarrow}_{L,\mathcal{A}} \langle a',l'\rangle}\ \textbf{Base(L)} \qquad \frac{\langle a,l\rangle \overset{\mathsf{P}}{\Longrightarrow}_{L,\mathcal{A}} \langle a',l'\rangle \qquad l' \geq \hat{l}}{\langle a,l\rangle \overset{\mathsf{P}}{\Longrightarrow}_{L,\mathcal{A}} \langle a',\hat{l}\rangle}\ \textbf{Weaken(L)}$$

$$\frac{\langle a,l\rangle \overset{\mathsf{P}_1}{\Longrightarrow}_{L,\mathcal{A}} \langle a_1,l_1\rangle \qquad \langle a_1,l_1\rangle \overset{\mathsf{P}_2}{\Longrightarrow}_{L,\mathcal{A}} \langle a_2,l_2\rangle}{\langle a,l\rangle \overset{\mathsf{P}_1;\mathsf{P}_2}{\Longrightarrow}_{L,\mathcal{A}} \langle a_2,l_2\rangle}\ \textbf{Seq(L)}$$

$$\frac{\langle a,l\rangle \overset{\mathsf{P}_1}{\Longrightarrow}_{L,\mathcal{A}} \langle a_1,l_1\rangle \qquad \langle a,l\rangle \overset{\mathsf{P}_2}{\Longrightarrow}_{L,\mathcal{A}} \langle a_2,l_2\rangle \qquad \textsc{LJoin}_{\mathcal{A}}(a_1,l_1,a_2,l_2,a',l')}{\langle a,l\rangle \overset{\mathsf{P}_1 \oplus \mathsf{P}_2}{\Longrightarrow}_{L,\mathcal{A}} \langle a',l'\rangle}\ \textbf{Join(L)}$$

$$\frac{\textsc{LimIter}_{\mathcal{A}}(a,l,[\![\mathsf{P}]\!]_C,[\![\mathsf{P}]\!]_{\mathcal{A}},(\lambda_n)_{n\in\mathbb{N}},a_\infty,\bar{l})}{\langle a,l\rangle \overset{\mathsf{P}^*}{\Longrightarrow}_{L,\mathcal{A}} \langle a_\infty,\bar{l}\rangle}\ \textbf{Iterate(L)} \qquad \frac{\langle \alpha(I),0\rangle \overset{\mathsf{P}}{\Longrightarrow}_{L,\mathcal{A}} \langle a,l\rangle}{l - \textsc{LBound}(\mathsf{P},I)_{\mathcal{A}}}\ \textbf{Program(L)}$$

Fig. 6. Pair-based forward rules for propagating lower bounds on abstract imprecision.

for "lower-bound witnesses." For example, in the the standard interval domain over real values Int, $\mathrm{Wit}_L([1,3],1) = \{[r,s] \mid 1 \leq r \leq s \leq 3, s-r \leq 2\}$. This witness set summarizes the possible exact abstract result represented by a pair (a,l) without retaining the intermediate concrete results.

The propagation rules use two auxiliary predicates. For a basic command e, the predicate

$$\textsc{LTrans}_{\mathcal{A}}(e,a,l,a',l') \iff a' = [\![e]\!]_{\mathcal{A}}(a) \ \wedge\ \forall \hat{a} \in \mathrm{Wit}_L(a,l).\ \delta_{\mathcal{A}}\left(\alpha([\![e]\!]_C(\gamma(\hat{a}))), a'\right) \geq l'.$$

requires a' to be the ordinary abstract transfer of a and l' to hold for every exact abstract result consistent with the input certificate. For nondeterministic choice, following prior join-bound constructions [3, 4], we define:

$$\textsc{LJoin}_{\mathcal{A}}(a_1,l_1,a_2,l_2,a',l') \iff a' = a_1 \vee_{\mathcal{A}} a_2 \ \wedge\ l' \leq \inf\{\delta_{\mathcal{A}}(\hat{a}_1 \vee_{\mathcal{A}} \hat{a}_2, a') \mid \hat{a}_i \in \mathrm{Wit}_L(a_i,l_i)\}.$$

Thus the join predicate needs only the two branch-output pairs: it considers all exact branch results consistent with their certified lower bounds and retains the smallest imprecision that must remain after the abstract join.

Example. In the interval domain Int, consider two pairs of abstract results and certified lower bounds: $([3,9],2)$ and $([11,14],1)$, $\textsc{LJoin}_{\mathcal{A}}([3,9],2,[11,14],1,[3,14],0)$ holds. This is because the certified losses can be witnessed by concrete intervals $[3,7] \subseteq [3,9]$ and $[12,14] \subseteq [11,14]$, and their join is $[3,14]$, which coincides with $[3,9] \vee_{\mathsf{Int}} [11,14]$. In contrast, for $([3,9],2)$ and $([5,7],1)$, the abstract join result is $[3,9] \vee_{\mathsf{Int}} [5,7] = [3,9]$. Joining $[5,7]$ cannot remove the loss 2 already certified for $[3,9]$, and $\textsc{LJoin}_{\mathcal{A}}([3,9],2,[5,7],1,[3,9],2)$ holds.

Figure 6 gives the pair-based forward rules. The base rule delegates a basic command to $\textsc{LTrans}_{\mathcal{A}}$, while weakening keeps the abstract result and lowers only its certificate. Sequential composition simply passes the output pair of P_1 to P_2; this is the main reason for making (a,l) explicit in the judgment. For nondeterministic choice, both branches start from the same input pair, and $\textsc{LJoin}_{\mathcal{A}}$ combines their output pairs. Finally, **Program(L)** initializes the analysis with $(\alpha(I),0)$ and exposes

the resulting certificate as $l - \textsc{LBound}(\mathsf{P}, I)_{\mathcal{A}}$. Operationally, the judgment therefore amounts to maintaining one abstract-result/lower-bound pair at every program point.

Loops require special care because the abstract result of P^* is a fixpoint rather than a finite iteration. The predicate LimIter is certificate-based: it does not compute the lower bound directly; in contrast, it verifies a proposed sequence of lower bounds against the final abstract loop invariant. A user or a domain-specific external procedure can propose such a sequence $(\lambda_n)_{n\in\mathbb{N}}$ together with a claimed limiting lower bound $\bar{l}$, and our logic checks that this sequence is valid:

$$\textsc{LimIter}_{\mathcal{A}}(a, l, [\![\mathsf{P}]\!]_C, [\![\mathsf{P}]\!]_{\mathcal{A}}, (\lambda_n)_{n\in\mathbb{N}}, a_\infty, \bar{l}) \iff a_\infty = [\![\mathsf{P}^*]\!]_{\mathcal{A}}(a) \wedge \liminf_{n\to\infty} \lambda_n \geq \bar{l}$$
$$\wedge\ \forall \hat{a} \in \mathrm{Wit}_L(a, l),\ \text{let } \hat{c}_n \triangleq \bigvee_C \{F_C^i(\gamma(\hat{a})) \mid 0 \leq i \leq n\}, \delta_{\mathcal{A}}\left(\alpha(\hat{c}_n), a_\infty\right) \geq \lambda_n\ (\forall n \in \mathbb{N}).$$

It reflects the lower-bound analog of a fixed-point invariant: it is not an invariant of reachability but an invariant of unavoidable distance from the abstract loop result. For a simpler verification of a proposed sequence $(\lambda_n)_{n\in\mathbb{N}}$, the following inductive criterion gives a usable way to establish LimIter. Assume that F_C is $\vee_C$-distributive, $\textsc{LimIter}_{\mathcal{A}}(a, l, [\![\mathsf{P}]\!]_C, [\![\mathsf{P}]\!]_{\mathcal{A}}, (\lambda_n)_{n\in\mathbb{N}}, a_\infty, \bar{l})$ holds if:

$$\forall a_I \in \mathrm{Wit}_L(a, l), \hat{a} \in \mathrm{Wit}_L(a_\infty, \lambda_n),\ \delta_{\mathcal{A}}\left(\alpha\left(\gamma(a_I) \vee_C F_C(\gamma(\hat{a}))\right), a_\infty\right) \geq \lambda_{n+1}\ (\forall n \in \mathbb{N}) \quad \text{(Induction)}$$

$$\forall a' \in \mathrm{Wit}_L(a, l), \delta_{\mathcal{A}}(a', a_\infty) \geq \lambda_0 \quad \text{(Initial Condition)} \qquad \liminf_{n\to\infty} \lambda_n \geq \bar{l} \quad \text{(Sequence Limit)}$$

This induction criterion says that any abstract value that is already far enough from the final invariant remains sufficiently far after one loop-body step and rejoining the initial input set.

Example. Consider domain Int equipped with the volume distance $Len([a, b]) \triangleq b - a$ and write the resulting distance as $\delta_{\mathsf{Int}}^{Len}$ as in Section 3.4. Let $(a, l) = ([-1, 1], 0)$, and let the loop body P be $\texttt{if } x \geq 0 \texttt{ then } x := 1 + x/2 \texttt{ else } x := -1 + x/2$. Its abstract loop invariant is $a_\infty = [-2, 2]$. We propose $\lambda_k \triangleq 2^{-k}$ and verify this sequence using the inductive criterion above.

It is trivial to check that $\lambda_0 = 1 \geq l = 0$. For the inductive step, fix $k \geq 0$. Since $\mathrm{Wit}_L([-1, 1], 0) = \{[-1, 1]\}$, a_I can only be $[-1, 1]$. Let $\hat{a} = [r, s] \in \mathrm{Wit}_L([-2, 2], \lambda_k)$. By the definition of the witness set, $s - r \leq 4 - \lambda_k$. Then we write $J \triangleq \alpha\left(\gamma(a_I) \vee_C F_C(\gamma(\hat{a}))\right)$. If $\hat{a}$ intersects both conditional branches, the convex interval enclosing its image under F_C has length $2 + \frac{s-r}{2} \leq 4 - \frac{\lambda_k}{2}$. This interval already contains $[-1, 1]$, so joining a_I does not increase its length. If $\hat{a}$ lies in only one branch, its image lies in either $[-2, -1]$ or $[1, 2]$; after joining $a_I \subseteq [-1, 1]$, the resulting interval has length at most 3. Since $\lambda_k \leq 1$, this is again at most $4 - \lambda_k/2$. Thus in all cases $\delta_{\mathsf{Int}}^{Len}(J, a_\infty) \geq \frac{\lambda_k}{2} = \lambda_{k+1}$, which completes the proposed certificate. Finally, $\liminf_k \lambda_k = 0$, so LimIter validates $\bar{l} = 0$ for this loop.

The rules in this section are meant to be used as a proof system: once a lower-bound judgment is derived, it should be interpreted as a certified quantitative fact about the actual imprecision on the fixed input set. The following theorem records this soundness property for the forward logic.

Theorem 5.1 (Soundness of the forward lower-bound logic). *If $l - \textit{LBound}(\mathsf{P}, I)_{\mathcal{A}}$ is derivable by the rules in Figure 6, then $l \leq \mathrm{err}_{\mathcal{A}}(\mathsf{P}, I)$.*

The theorem above treats every application of **Iterate(L)** as relying on a certified LimIter. The only remaining loop-specific obligation is to justify the simpler induction criterion used above. The next theorem states that this criterion is sufficient for establishing the LimIter predicate.

Theorem 5.2 (Soundness of the LimIter induction criterion). *Let $F_C = [\![\mathsf{P}]\!]_C$ be $\vee_C$-distributive and let $F_{\mathcal{A}} = [\![\mathsf{P}]\!]_{\mathcal{A}}$. Suppose $a_\infty = [\![\mathsf{P}^*]\!]_{\mathcal{A}}(a)$. If the initial condition, induction condition, and sequence-limit condition in the induction criterion above all hold for $(\lambda_n)_{n\in\mathbb{N}}$ and $\bar{l}$, then $\textsc{LimIter}_{\mathcal{A}}(a, l, F_C, F_{\mathcal{A}}, (\lambda_n)_{n\in\mathbb{N}}, a_\infty, \bar{l})$ holds.*

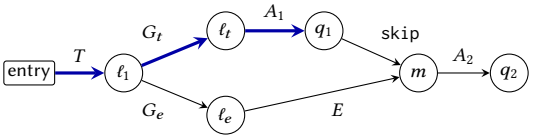


Fig. 7. Summary CFG of the overview program. Here T is the affine transfer, $G_t = (x \geq 0)?$, $G_e = (x < 0)?$, $E = (x := x - \frac{1}{2})$, and A_i is the assertion ending at q_i. The thick blue edges form the relevant subgraph for q_1; the whole graph forms the relevant subgraph for q_2.

5.2 Backward Learning from Posterior Information

The forward logic in Section 5.1 can be conservative, as its rules propagate only abstract results and previously certified lower bounds as shown in Section 2.3. Posterior information offers a second source of evidence naturally in interactive uses of static analysis: developers often inspect a subset of reported alarms during triage, suppress alarms confirmed to be false positives, or validate alarms through tests and bug reports. The key idea is to treat such observations as semantic constraints on the set of feasible concrete behaviors. Note that the mechanism discussed in this subsection is optional: without such feedback, the framework still reports the initial lower-bound certificate.

To make the source of posterior information explicit, we extend the basic commands from Section 3.2 with assertions, following the setting of prior work on learning-based program analysis [20]: $\mathit{BaseCmd}^{+} \triangleq \mathit{BaseCmd} \cup \{\texttt{assert}(R) \mid R \in C\}$. Note that assertions do not change both concrete and abstract values, while their only additional behavior is to report an alarm at the program location immediately after the assertion whenever $\gamma(a) \not\subseteq R$ for the reaching abstract result a.

Control-flow graphs and program locations. To make the execution structure explicit and expose how posterior information propagates between program points, we associate each program P with an edge-labeled control-flow graph $\mathcal{G}_{\mathsf{P}} = (V_{\mathsf{P}}, E_{\mathsf{P}}, \mathsf{entry})$ where $E_{\mathsf{P}} \subseteq V_{\mathsf{P}} \times \mathsf{Cmd}_{\mathsf{P}} \times V_{\mathsf{P}}$. The distinguished node entry denotes the start of execution, and every other node is a program location. An edge $(p, e, r) \in E_{\mathsf{P}}$ executes command e from location p and reaches its output location r, and we use $e^{p \to r}$ named *located edge* to denote this tuple in the following text.

The graph is constructed compositionally from the program syntax [13, 19]. The construction is illustrated for the overview program in Figure 7: commands label edges, while the nodes denote the locations at which abstract and posterior values are maintained. Here the then branch ends with an assertion, so a semantic no-op edge labeled by `skip` connects its assertion site q_1 to the merge location m. For simplicity, we restrict that assertions are not allowed inside loop bodies. Each Kleene iteration P_0^* is collapsed into one synthetic basic edge $\mathsf{loop}(\mathsf{P}_0^*)$, whose concrete and abstract transfers are $[\![\mathsf{P}_0^*]\!]_C$ and $[\![\mathsf{P}_0^*]\!]_{\mathcal{A}}$, respectively. Hence $\mathsf{Cmd}_{\mathsf{P}} = \mathit{BaseCmd}^{+} \cup \{\mathsf{loop}(\mathsf{P}_0^*) \mid \mathsf{P}_0^* \text{ occurs in } \mathsf{P}\}$. Collapsing every loop makes $\mathcal{G}_{\mathsf{P}}$ a directed acyclic graph, where internal loop locations are opaque to posterior learning. An assertion occurrence is represented by an edge $\texttt{assert}(R)^{p \to q}$ We call q its *assertion site* and also write the edge label as $\texttt{assert}_q(R)$.

Relevant subgraphs and path programs. Let p be a program location and let q be an assertion site reachable from p. The relevant subgraph from p to q contains exactly the nodes and edges that occur on at least one directed path from p to q: $V_{p \rightsquigarrow q} \triangleq \{v \in V_{\mathsf{P}} \mid p \rightsquigarrow_{\mathcal{G}_{\mathsf{P}}}^{*} v \wedge v \rightsquigarrow_{\mathcal{G}_{\mathsf{P}}}^{*} q\}$, $E_{p \rightsquigarrow q} \triangleq \{(u, e, v) \in E_{\mathsf{P}} \mid u, v \in V_{p \rightsquigarrow q}\}$, $\mathcal{G}_{p \rightsquigarrow q} \triangleq (V_{p \rightsquigarrow q}, E_{p \rightsquigarrow q}, p)$. This subgraph contains all program paths from p to q, and it can be captured by a regular program expression in our syntax as previous work did [21, 22]. Its atomic terms are located edges rather than unlocated basic

commands: $\Pi ::= e^{u\to v} \mid \Pi_1;\Pi_2 \mid \Pi_1 \oplus \Pi_2$. Locations are annotations, so $[\![e^{u\to v}]\!] = [\![e]\!]$. We denote the expression reconstructed from $\mathcal{G}_{p\rightsquigarrow q}$ by $\Pi_{p\rightsquigarrow q}$ and call it the *path program* from p to q.

For the overview CFG in Figure 7, the relevant subgraph for the first alarm yields $\Pi_{\mathsf{entry}\rightsquigarrow q_1} = T^{\mathsf{entry}\to\ell_1}; G_t^{\ell_1\to\ell_t}; A_1^{\ell_t\to q_1}$, whereas the entire graph is relevant to the second alarm and yields $\Pi_{\mathsf{entry}\rightsquigarrow q_2} = T^{\mathsf{entry}\to\ell_1}; \Big(\big(G_t^{\ell_1\to\ell_t}; A_1^{\ell_t\to q_1}; \mathtt{skip}^{q_1\to m}\big) \oplus \big(G_e^{\ell_1\to\ell_e}; E^{\ell_e\to m}\big)\Big); A_2^{m\to q_2}$. The second path program therefore retains both branches and their merge, rather than selecting one execution trace.

For a fixed input set $I \in \mathcal{C}$, define the concrete and abstract results at an assertion site q by $C_q(I) \triangleq [\![\Pi_{\mathsf{entry}\rightsquigarrow q}]\!]_{\mathcal{C}}(I)$ and $A_q(I) \triangleq [\![\Pi_{\mathsf{entry}\rightsquigarrow q}]\!]_{\mathcal{A}}(\alpha(I))$. We say that $\mathtt{assert}_q(R)$ is a false alarm on I when $\textsc{FalseAlarm}(q,R,I) \Leftrightarrow \gamma(A_q(I)) \not\subseteq R \ \wedge\ C_q(I) \subseteq R$. The first conjunct records the abstract alarm, while the second records the posterior validation of its specification. If $C_q(I) \not\subseteq R$, the alarm is real and supplies no posterior restriction.

Weakest preconditions and backward annotations. We use weakest preconditions to propagate posterior feedback backward from an assertion site, thereby characterizing how the feedback restricts the outputs at earlier program points. We write predicates and their corresponding concrete sets interchangeably. For $R, Q \in \mathcal{C}$, the weakest precondition is characterized by $Q \subseteq \mathsf{wp}(\mathsf{P}, R) \Leftrightarrow [\![\mathsf{P}]\!]_{\mathcal{C}}(Q) \subseteq R$. For a summary loop edge, $\mathsf{wp}(\mathsf{loop}(\mathsf{P}_0^*), R)$ means $\mathsf{wp}(\mathsf{P}_0^*, R)$.

A false alarm at q restricts not only the output of $\Pi_{\mathsf{entry}\rightsquigarrow q}$ but also the outputs of earlier edges in its relevant subgraph. We define $\mathsf{Bwd}(\Pi, R) \subseteq V_{\mathsf{P}} \times \mathcal{C}$ so that $(r, S) \in \mathsf{Bwd}(\Pi, R)$ requires the concrete output at location r to satisfy S. On the acyclic path-program structure,

$$\begin{aligned} \mathsf{Bwd}(e^{p\to r}, R) &= \{(r, R)\}, \\ \mathsf{Bwd}(\Pi_1;\Pi_2, R) &= \mathsf{Bwd}(\Pi_2, R) \cup \mathsf{Bwd}(\Pi_1, \mathsf{wp}(\Pi_2, R)), \\ \mathsf{Bwd}(\Pi_1 \oplus \Pi_2, R) &= \mathsf{Bwd}(\Pi_1, R) \cup \mathsf{Bwd}(\Pi_2, R). \end{aligned}$$

A loop summary is handled by the first clause as one atomic edge, and its effect is included by wp. Since the path program contains every route from entry to q, a validated false alarm guarantees that each reachable result annotated in this way satisfies its associated restriction.

Example. For the path program of alarm q_1 in Figure 7: $\Pi_{\mathsf{entry}\rightsquigarrow q_1} = T^{\mathsf{entry}\to\ell_1}; G_t^{\ell_1\to\ell_t}; A_1^{\ell_t\to q_1}$, where G_t is the guard $x \geq 0$ and A_1 is $\mathtt{assert}(R_1)$ with $R_1 = \{(x,y) \mid y - x \geq -3\}$. Since assertions are observational, $\mathsf{wp}(A_1, R_1) = R_1$, and $\mathsf{Bwd}(\Pi_{\mathsf{entry}\rightsquigarrow q_1}, R_1) = \{(q_1, R_1), (\ell_t, R_1), (\ell_1, \mathsf{wp}(G_t; A_1, R_1))\}$, where $\mathsf{wp}(G_t; A_1, R_1) = \{(x,y) \mid x < 0\} \cup \{(x,y) \mid y - x \geq -3\}$.

Backward posterior update. We organize posterior learning as the procedure in Algorithm 1, which processes one validated feedback item at a time. Line 2 constructs the relevant subgraph and its path program, and Line 3 applies Bwd to obtain the posterior restrictions at the affected program locations. The pair maintained by the forward analysis is extended at every location r to a triple initialized as (a_r, l_r, w_r) with $w_r \leftarrow a_r$. Here a_r is the original abstract result and is never changed, l_r is the certified lower bound, and $w_r \in \mathcal{A}$ persistently stores posterior information.

Our algorithm processes one validated feedback (q, R) at a time. Lines 4–6 of Algorithm 1 apply the resulting restrictions. For each annotated location r, let $S_r^{q,R} \triangleq \bigcap\{S \mid (r,S) \in \mathsf{Bwd}(\Pi_{\mathsf{entry}\rightsquigarrow q}, R)\}$. If no annotation is generated at r, no update is performed there. Otherwise, $w_r \leftarrow w_r \wedge_{\mathcal{A}} \alpha(S_r^{q,R})$. The intersection combines restrictions produced by multiple occurrences of a location in the path expression, while the abstract meet combines this item with all earlier feedback.

Example. For the feedback (q_1, R_1) in our instance of Section 2, the backward annotations give $S_{q_1}^{q_1,R_1} = S_{\ell_t}^{q_1,R_1} = R_1$, and $S_{\ell_1}^{q_1,R_1} = \{(x,y) \mid x < 0\} \cup \{(x,y) \mid y - x \geq -3\}$. The backward update changes only the w-component: since $a_{\ell_t} = a_{q_1} = a_{\mathsf{then}} = [0, 3.75] \times [-3, 3]$, $w_{\ell_t} = w_{q_1} = a_{\mathsf{then}} \wedge_{\mathsf{Poly}} \mathsf{Conv}(R_1) = a_{\mathsf{then}} \cap R_1$, as shown in the right panel of Figure 4. However, $\mathsf{Conv}(S_{\ell_1}^{q_1,R_1}) = \mathbb{R}^2$

Algorithm 1 Updating posterior triples from one false alarm

Require: Fixed input set I, summary CFG $\mathcal{G}_{\mathsf{P}}$, triples (a_r, l_r, w_r), and feedback (q, R)

1: Check FalseAlarm(q, R, I)
2: Construct $\mathcal{G}_{\mathsf{entry}\rightsquigarrow q}$ and a corresponding path program $\Pi_{\mathsf{entry}\rightsquigarrow q}$
3: $\mathcal{B} \leftarrow \mathsf{Bwd}(\Pi_{\mathsf{entry}\rightsquigarrow q}, R)$
4: **for all** locations r occurring in $\mathcal{B}$ **do**
5: $\quad S_r^{q,R} \leftarrow \bigcap\{S \mid (r, S) \in \mathcal{B}\}$
6: $\quad w_r \leftarrow w_r \wedge_{\mathcal{A}} \alpha(S_r^{q,R})$
7: **for all** non-entry locations r in a topological order of $\mathcal{G}_{\mathsf{P}}$ **do**
8: $\quad \widehat{w}_r \leftarrow \bigvee_{(p,e,r)\in E_{\mathsf{P}}} [\![e]\!]_{\mathcal{A}}(w_p)$
9: $\quad w_r \leftarrow w_r \wedge_{\mathcal{A}} \widehat{w}_r$
10: **for all** locations r **do**
11: $\quad l_r \leftarrow \max\{l_r, \delta_{\mathcal{A}}(w_r, a_r)\}$
12: **return** the updated triples (a_r, l_r, w_r)

and $w_{\ell_1} = a_{\ell_1} \wedge_{\mathsf{Poly}} \mathrm{Conv}(S_{\ell_1}^{q_1,R_1}) = a_{\ell_1}$. It shows that the backward update does not refine w_{ℓ_1} in Poly, nor does it directly constrain the states entering the else branch. Its later effect on m and q_2 arises only through subsequent forward propagation from the refined then-branch posterior.

The backward update only affects locations in the relevant subgraph $\mathcal{G}_{\mathsf{entry}\rightsquigarrow q}$. We then propagate this posterior information to the remaining CFG locations. Lines 7–9 of Algorithm 1 perform this process. Let $\prec$ be a topological order of $\mathcal{G}_{\mathsf{P}}$. For each non-entry location r, define $\widehat{w}_r \triangleq \bigvee_{(p,e,r)\in E_{\mathsf{P}}} [\![e]\!]_{\mathcal{A}}(w_p)$, and visit the locations once in order $\prec$, applying $w_r \leftarrow w_r \wedge_{\mathcal{A}} \widehat{w}_r$. All incoming values are joined before this meet. Since every predecessor of r occurs earlier in $\prec$, one pass propagates the current feedback through all downstream locations.

Finally, Line 11 of Algorithm 1 updates each affected lower bound by $l_r \leftarrow \max\{l_r, \delta_{\mathcal{A}}(w_r, a_r)\}$. The maximum also retains any stronger certificate obtained before this feedback round.

Example. In the example of Section 2, since w_{ℓ_1} and w_{ℓ_e} are unchanged by the backward update, the effect reaches later locations m and q_2 only through forward propagation in Lines 7–9 of Algorithm 1. At the merge location m, $\widehat{w}_m = [\![\mathtt{skip}]\!]_{\mathsf{Poly}}(w_{q_1}) \vee_{\mathsf{Poly}} [\![E]\!]_{\mathsf{Poly}}(w_{\ell_e})$, and the posterior component is updated by $w_m \leftarrow w_m \wedge_{\mathsf{Poly}} \widehat{w}_m$. Since A_2 is an assertion and does not change the abstract value, $w_{q_2} = w_m$, which is shown in the right panel of Figure 4. The lower-bound update gives $l_{q_2} \leftarrow \max\{0, \delta_{\mathcal{A}}(w'_{q_2}, a_{q_2})\} \approx 39 - 31.969 = 7.031$. Thus the posterior feedback from the false alarm at q_1 raises the lower bound at the later alarm site q_2 from 0 to 7.031.

Soundness of the posterior update follows from the invariant in the theorem below:

Theorem 5.3 (Soundness of posterior lower-bound learning). *For the fixed input set I, define the invariant at every location r by $\mathsf{Inv}_I(r) \Leftrightarrow \alpha(C_r(I)) \sqsubseteq_{\mathcal{A}} w_r \sqsubseteq_{\mathcal{A}} a_r \wedge l_r \le \delta_{\mathcal{A}}\big(\alpha(C_r(I)), a_r\big)$. Assume $\mathsf{Inv}_I(r)$ holds at every location, every edge transformer in $\mathcal{G}_{\mathsf{P}}$ is sound and monotone, and (q, R) satisfies FalseAlarm(q, R, I). After applying Algorithm 1, $\mathsf{Inv}_I(r)$ still holds at every location.*

Consequently, after any finite sequence of validated feedback items, every l_r remains a sound lower bound; in particular, the bound at the program exit is at most $\mathrm{err}_{\mathcal{A}}(\mathsf{P}, I)$.

5.3 Two-Dimensional Numerical Instantiation

We now instantiate the lower-bound propagation in the two-dimensional numerical setting. We focus on the fragment whose basic commands are generated by $c ::= \boldsymbol{v} := A\boldsymbol{v} + b \mid G? \mid \mathtt{assert}_q(R)$, where $\boldsymbol{v} = (x, y)^\top$ is a two-dimensional vector of variables, $A \in \mathbb{R}^{2\times 2}$, $b \in \mathbb{R}^2$, and $G, R \subseteq \mathbb{R}^2$ are

$$\frac{K' = [\![\boldsymbol{v} := A\boldsymbol{v} + b]\!]_{\mathsf{Poly}}(K)}{\textsc{LTrans}_{\mathsf{Poly}}(\boldsymbol{v} := A\boldsymbol{v} + b, K, l, K', |\det A| \cdot l)}\ \textsc{Affine-LTrans}_{\mathsf{Poly}}$$

$$\frac{K' = \mathrm{Conv}(K \cap G) \qquad \Delta_G(K) \triangleq \mathrm{Vol}(K) - \mathrm{Vol}(K')}{\textsc{LTrans}_{\mathsf{Poly}}(G?, K, l, K', \max\{0, l - \Delta_G(K)\})}\ \textsc{Guard-LTrans}_{\mathsf{Poly}}$$

Fig. 8. Numerical instances of $\textsc{LTrans}_{\mathsf{Poly}}$ for two-dimensional polygons.

regions. The guard command $G?$ keeps exactly those points that lie in G; the assertion command checks the specification region R. The concrete domain is $C = \mathcal{P}(\mathbb{R}^2)$, and the ideal abstract domain is the domain Poly of convex polygons ordered by inclusion. The abstraction α_{Poly} maps a set of points to its convex hull. In this subsection, the distance is area-based: $\delta_{\mathsf{Poly}}(K', K) \triangleq \mathrm{Vol}(K) - \mathrm{Vol}(K')$, where Vol denotes two-dimensional area. Thus a lower bound $l - \textsc{LBound}(\mathsf{P}, I)_{\mathsf{Poly}}$ states that at least area l of the implemented abstract result consists of spurious numerical space introduced by over-approximation: this area is present in the abstract result but absent from the exact convex hull of the reachable concrete output.

We consider a practical setting in which the abstract domain remains Poly throughout the analysis, but the implementation may simplify overly complex polygons as in Section 2. In particular, when a polygon exceeds the chosen complexity threshold, for example its number of support edges exceeds a bound (5 in Section 2), it is over-approximated by its least axis-aligned interval box, i.e., the rectangle obtained by maintaining one interval for each coordinate. This is the point at which unavoidable abstract error is introduced in our setting, and we write $\rho(K) = K$ when K is within the complexity threshold, and $\rho(K) = \mathrm{Box}(K)$ otherwise, where $\mathrm{Box}(K)$ is the least axis-aligned rectangle containing K. If τ^{Poly} is the bca in Poly, then the implemented transfer is $\tau^{\mathsf{act}} = \rho \circ \tau^{\mathsf{Poly}}$, and this simplification gives a directly checkable lower-bound fact: $\mathrm{Loss}_\rho(K) \triangleq \delta_{\mathsf{Poly}}(K, \rho(K)) = \mathrm{Vol}(\rho(K)) - \mathrm{Vol}(K)$. This quantity is the area introduced solely by the implementation's complexity control. It can be used as a domain-specific instance of **Base(L)**.

After the base loss is certified, the generic **Seq(L)** rule can in principle propagate it through later commands, but doing so may require reasoning over the whole witness family $\mathrm{Wit}_L(a_m, l)$. In this numerical setting, our particular rules avoid this enumeration by propagating area loss directly through common numerical transfers, while still producing a sound lower bound.

The affine $\textsc{LTrans}_{\mathsf{Poly}}$ instance is exact for the area distance. For the command $\boldsymbol{v} := A\boldsymbol{v} + b$, the only effect on area is that the linear part scales area by $|\det A|$. Writing $T_A(u) = Au + b$, we have $\delta_{\mathsf{Poly}}(T_A(K'), T_A(K)) = |\det A| \cdot \delta_{\mathsf{Poly}}(K', K)$, and hence an area loss lower bound l becomes $|\det A| \cdot l$ after the affine assignment.

For a guard, $K' = \mathrm{Conv}(K \cap G)$ represents applying $G?$ to K and closing the result back into the polyhedral domain. The quantity $\Delta_G(K) \triangleq \mathrm{Vol}(K) - \mathrm{Vol}(K')$ is the area removed from the current abstract result. Intuitively, the guard can discharge at most this much previously certified spurious area, so at least $\max\{0, l - \Delta_G(K)\}$ remains.

Example. In the example of Section 2, before the if guard we have $a = [-3, 3] \times [-3, 3]$, $l = 16$. For the guard $G = \{(x, y) \mid x \geq 0\}$, the abstract result is $\mathrm{Conv}(a \cap G) = [0, 3] \times [-3, 3]$, whose area is 18, while $\mathrm{Area}(a) = 36$. Therefore, the guard can remove area $\Delta_G(a) = 36 - 18 = 18$ at most, and the post-guard lower bound is $l' = \max\{0, l - \Delta_G(a)\} = \max\{0, 16 - 18\} = 0$.

The soundness guarantee of the numerical lower-bound instantiation is summarized as follows.

Theorem 5.4 (Soundness of the numerical lower-bound instantiation). *If the predicate* $l - \mathit{LBound}(\mathsf{P}, I)_{\mathsf{Poly}}$ *is derived using* ***Base(L)*** *together with the* $\mathit{LTrans}_{\mathsf{Poly}}$ *instances in Figure 8, then* $l \leq \mathsf{err}_{\mathsf{Poly}}(\mathsf{P}, I)$.

6 Upper-Bound Reasoning

The previous section certifies the lower bound of the unavoidable imprecision. We now turn to the upper-bound side. Our focus is to improve the choice of concrete states retained for upper-bound reasoning by replacing the fixed generator choice with configurable representative states.

6.1 From Generators to Representative States

We begin by defining a family of upper bounds parameterized by a set of retained concrete states.

Definition 6.1 (Representative-based upper bound)**.** For a concrete input set $I \in C$, a program P, and representatives $R \subseteq I$, define $U_R(\mathsf{P}, I) \triangleq \delta_{\mathcal{A}}\left(\alpha\left(\llbracket \mathsf{P} \rrbracket_C(R)\right), \llbracket \mathsf{P} \rrbracket_{\mathcal{A}}(\alpha(I))\right)$.

A particularly economical choice, adopted by the generator-based EPL of Campion et al. [4], is $R = g$, where $g \subseteq I$ is a generator that is a minimal element satisfying $\alpha(g) = \alpha(I)$. Generators provide minimal concrete witnesses for the current abstract property. They substantially reduce the search space, and under the conditions identified by Campion et al., reasoning over generators can be lifted to larger sets of states. Thus, using g is an effective way to preserve the current abstraction while retaining very few states.

This compactness, however, has a limitation: preserving the current abstract element does not guarantee that g remains representative after a later restriction. A guard may remove generator states while retaining interior states that were omitted from g, making $U_g(\mathsf{P}, I)$ unnecessarily loose.

Example. Consider the interval domain Int, the input set $I = \{-1, 0, 1\}$, and its generator $g = \{-1, 1\}$, for which $\alpha_{\mathsf{Int}}(I) = \alpha_{\mathsf{Int}}(g) = [-1, 1]$. Let P be the guard $(x \geq 0)?$. Then $\llbracket \mathsf{P} \rrbracket_C(I) = \{0, 1\}$, $\llbracket \mathsf{P} \rrbracket_C(g) = \{1\}$, and $\llbracket \mathsf{P} \rrbracket_{\mathcal{A}}([-1, 1]) = [0, 1]$. The interval analysis is exact on I, so its true imprecision is zero. Under the volume distance $\delta^{Len}_{\mathsf{Int}}$ defined in the loop example of Section 5.1, retaining only the generator gives $U_g(\mathsf{P}, I) = \delta^{Len}_{\mathsf{Int}}([1, 1], [0, 1]) = 1$. The state 0 is needed to represent the restricted interval, but it is an interior state absent from the original generator.

This example motivates replacing the fixed choice $R = g$ with a configurable set of representative states $g \subseteq R \subseteq I$. Such a set retains the generator's exact representation of the current abstract input while preserving additional concrete information for later program constructs. The following result justifies this generalization.

Theorem 6.1 (Soundness and monotonicity). *For every* $R \subseteq I$, $\mathsf{err}_{\mathcal{A}}(\mathsf{P}, I) \leq U_R(\mathsf{P}, I)$. *Moreover, if* $R_1 \subseteq R_2 \subseteq I$, *then* $U_{R_2}(\mathsf{P}, I) \leq U_{R_1}(\mathsf{P}, I)$.

The generator-only method is recovered by taking $R = g$, while $R = I$ recovers the exact imprecision. Our goal is to retain substantially fewer states than I while obtaining a meaningfully tighter upper bound than $U_g(\mathsf{P}, I)$.

6.2 Guidance for Selecting Representative States

Theorem 6.1 shows that retaining more concrete states cannot worsen the upper bound. However, keeping all of I defeats the purpose of a compact representation. We therefore need guidance for choosing a small set R that preserves the concrete information most likely to matter later. We retain a generator $g \subseteq R$ to guarantee $\alpha(R) = \alpha(I)$, and use the remaining budget to retain more concrete information under future restrictions.

Probability over future restrictions. Since complex programs can impose numerous different restrictions on intermediate states, requiring a single finite representative set to remain precise under every possible restriction would be too strong. We therefore use probability only to formalize how often a precision guarantee holds over the restrictions that may arise later.

A measurable space is a pair $(\mathsf{X}, \mathcal{X})$, where X is a nonempty set called sample space and $\mathcal{X}$ is a σ-algebra over X: it contains $\emptyset$ and is closed under complement and countable union. A measure on $(\mathsf{X}, \mathcal{X})$ is a function $\mu : \mathcal{X} \to [0, \infty]$ such that $\mu(\emptyset) = 0$ and, for every pairwise-disjoint countable family $A_1, A_2, \ldots \in \mathcal{X}$, $\mu\Big(\bigcup_i A_i\Big) = \sum_i \mu(A_i)$. When $\mu(\mathsf{X}) = 1$, we call μ a distribution on X; we write $Z \sim \mu$ for sampling from μ and let $\mathcal{D}(\mathsf{X})$ denote the set of such distributions. A function $f : (\mathsf{X}_1, \mathcal{X}_1) \to (\mathsf{X}_2, \mathcal{X}_2)$ is measurable if $f^{-1}(A) \in \mathcal{X}_1$ for every $A \in \mathcal{X}_2$. For a measurable event $A \in \mathcal{X}$, $\Pr_{Z\sim\mu}[Z \in A] \triangleq \mu(A)$. For a measurable predicate Ψ, we write $\Pr_{Z\sim\mu}[\Psi(Z)]$ for the probability of its satisfying set. If $f : \mathsf{X} \to \mathbb{R}^{\infty}_{\geq 0}$ is measurable, its expectation is $\mathbb{E}_{Z\sim\mu}[f(Z)] \triangleq \int_{\mathsf{X}} f(z)\, d\mu(z)$. A distribution is supported on $S \in \mathcal{X}$ when $\mu(S) = 1$.

Example: Borel space over numerical states. Consider n-variable programs, their states can be regarded as vectors in $\Sigma = \mathbb{R}^n$. A standard choice of measurable space is the Borel space: $(\mathbb{R}^n, \mathcal{B}(\mathbb{R}^n))$, where $\mathcal{B}(\mathbb{R}^n)$ is the Borel σ-algebra generated by the open subsets of $\mathbb{R}^n$. A future restriction such as a linear guard $a^\top x \geq t$ then denotes a measurable restriction $G_t = \{x \in \mathbb{R}^n \mid a^\top x \geq t\} \in \mathcal{B}(\mathbb{R}^n)$.

The selection objective. Let $\mathcal{G}_I$ be a family of possible restrictions on I, where each $G \in \mathcal{G}_I$ denotes the states that may be preserved by a future program construct. We equip this family with a distribution $\mu_I \in \mathcal{D}(\mathcal{G}_I)$ describing how likely different future restrictions are. The probability does not change the concrete program semantics; it lets us state that, for most restrictions that may arise later, the representatives selected now should still preserve the abstract meaning of the restricted concrete states as much as possible.

Definition 6.2 (Restriction-stable representative set)**.** A set $R \subseteq I$ is (ε, δ)-restriction-stable with respect to $(\alpha, \delta_{\mathcal{A}}, \mathcal{G}_I, \mu_I)$ if $\Pr_{G\sim\mu_I}\left[\delta_{\mathcal{A}}\big(\alpha(R \cap G), \alpha(I \cap G)\big) \leq \varepsilon\right] \geq 1 - \delta$.

The distance $\delta_{\mathcal{A}}\big(\alpha(R \cap G), \alpha(I \cap G)\big)$ measures the abstract information lost when $I \cap G$ is approximated using only the surviving representatives $R \cap G$. The probability is taken over future restrictions. Thus, the definition expresses the following requirement: for most restrictions that may arise later, the representatives selected now should still preserve the abstract meaning of the restricted concrete states as much as possible. In this sense, additional representatives provide redundancy for future state-filtering operations, as they make the current abstraction more stable under subsequent semantic restrictions.

Example. Recall the example in Section 2. As a simple family of possible future restrictions, consider vertical linear guards $\mathcal{G}_I = \{G_t \mid -5 \leq t \leq 5\}, G_t \triangleq \{(x, y) \in \mathbb{R}^2 \mid x \geq t\}$, where t is sampled from the uniform distribution $\mathcal{U}([-5, 5])$. Our selected cover (denoted as R here) in Section 2.4 satisfies $\Pr_{t\sim\mathcal{U}([-5,5])}\left[\delta^{\text{Area}}_{\text{Poly}}\big(\text{Conv}(R \cap G_t), \text{Conv}(I \cap G_t)\big) \leq 3\right] \geq 1 - \frac{3\sqrt{2}}{40} \approx 0.894$, since only when $t \in (-2\sqrt{2}, -\frac{7\sqrt{2}}{4}] \cup (-\frac{3\sqrt{2}}{4}, -\frac{\sqrt{2}}{2}] \cup (\frac{\sqrt{2}}{2}, \frac{3\sqrt{2}}{4}]$ does the inequality ≤ 3 not hold.

A concrete selection metric. However, the above definition specifies the semantic goal of selection, but does not yet say how to construct R. To obtain an actionable selection strategy, we equip all the concrete states with a task-dependent distance or dissimilarity measure $d_\Sigma : \Sigma \times \Sigma \to \mathbb{R}^{\infty}_{\geq 0}$. It quantifies state similarity: a retained state can represent nearby omitted states, so a good set R should place every state in I close to some representative. This turns the goal of seeking a representative set for I into a numerical selection criterion.

Definition 6.3 (h-cover)**.** A set $R \subseteq I$ is an h-cover of I with respect to d_Σ if $\forall \sigma \in I.\ \exists r \in R.\ d_\Sigma(\sigma, r) \leq h$.

The smaller h is, the more densely R covers I under the concrete selection metric. Thus, under a fixed state budget, representative selection becomes a covering problem: choose states that make the worst represented part of I as close as possible to R.

Example. In the running example of Section 2, we use the Euclidean metric $d_\Sigma((x_1, y_1), (x_2, y_2)) \triangleq \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$, while our selected representative set contains the six generator points and five additional interior points, which achieves a 1.5-cover of the input set I.

Beyond the Euclidean instance above, the selection metric can be chosen according to the program state. For other numerical states, d_Σ may be extended to a weighted norm or a scale-normalized distance between valuations; while for heap- or graph-shaped states like in pointer analysis, it may compare aliasing relations or reachable structure.

Additionally, it is important to distinguish d_Σ from the abstract imprecision distance $\delta_{\mathcal{A}}$. The former is a concrete selection metric, while the latter is the semantic quantity that our upper-bound estimation ultimately aims to control. They play different roles: d_Σ *guides representative selection*, whereas $\delta_{\mathcal{A}}$ *evaluates the abstract loss caused by using those representatives.* In general, d_Σ need not be induced by $\delta_{\mathcal{A}}$, and $\delta_{\mathcal{A}}$ need not be derived from d_Σ. The connection between them is established only through the compatibility condition introduced below.

Connecting selection to abstract precision. As discussed above, the h-cover condition is purely concrete. By itself, it does not imply a bound on $\delta_{\mathcal{A}}\big(\alpha(R \cap G), \alpha(I \cap G)\big)$, and hence we introduce the metric–abstraction compatibility to make explicit when concrete coverage can be converted into an abstract-domain guarantee:

Definition 6.4 (Metric–abstraction compatibility)**.** Given an abstraction α, a restriction family $\mathcal{G}_I$ with distribution μ_I, an abstract distance $\delta_{\mathcal{A}}$, and a selection metric d_Σ, we say that d_Σ and $\delta_{\mathcal{A}}$ are *compatible* with respect to $(\alpha, \mathcal{G}_I, \mu_I)$ if there exists a function $\Omega_{I,\mathcal{G}}(h, \delta)$ such that every h-cover $R \subseteq I$ satisfies

$$\Pr_{G \sim \mu_I}\left[\delta_{\mathcal{A}}\big(\alpha(R \cap G), \alpha(I \cap G)\big) \leq \Omega_{I,\mathcal{G}}(h, \delta)\right] \geq 1 - \delta.$$

Moreover, $\Omega_{I,\mathcal{G}}(h, \delta)$ is monotone in h and tends to zero as $h \to 0$ for fixed $I, \mathcal{G}_I$, and δ, whenever the restriction family and abstraction are sufficiently regular.

When d_Σ and $\delta_{\mathcal{A}}$ are compatible, reducing the concrete covering radius provably reduces the abstract loss for most restrictions. It therefore justifies using d_Σ to select states even though the final precision is measured by $\delta_{\mathcal{A}}$.

When metric–abstraction compatibility holds, obtaining an (ε, δ)-restriction-stable set reduces to choosing h such that $\Omega_{I,\mathcal{G}}(h, \delta) \leq \varepsilon$ and constructing an h-cover of I. Our two-dimensional numerical setting in Section 5.3 next provides one concrete instance where this compatibility condition can be explicitly proved. The next subsection returns to this setting from the upper-bound side: Euclidean coverage is used to control area loss after future guards.

6.3 Two-Dimensional Numerical Instantiation

We continue with the setting of Section 5.3. Thus $I \subseteq \mathbb{R}^2$ is a finite set of concrete states, $K \triangleq \alpha_{\mathsf{Poly}}(I)$ is its convex hull, and abstract loss is measured by area difference. For the simplicity of discussion, we restrict the G from Definition 6.2 to be instantiated as the linear guard: $G(n, t) = \{x \in \mathbb{R}^2 \mid n^\top x \geq t\}$ where $\|n\|_2 = 1$. Here this G is the same guard region as the G in the command $G?$ of Section 5.3: it denotes the subset of states that survive the guard.

Generators are exactly the support points of the convex hull: retaining all hull vertices guarantees that the current abstraction is preserved. However, support points alone may be insufficient after a guard, because a guard can remove boundary representatives while preserving interior points. Since in this setting it is natural to measure distances between concrete states using the Euclidean metric $d_\Sigma(u,v) \triangleq \|u-v\|_2$, we consider representative sets that both preserve the current abstraction and form an h-cover under this distance.

Near-uniform threshold assumption. Fix a direction $n \in \mathbb{S}^1$ and define the effective cutting interval of K by $a_n \triangleq \min_{x\in I} n^\top x$, $b_n \triangleq \max_{x\in I} n^\top x$, $w_n \triangleq b_n - a_n$. We assume $w_n > 0$ and that the threshold t is sampled from a distribution μ_n supported on $[a_n, b_n]$. Let λ denote the one-dimensional Lebesgue measure. For some constant $c_{\text{unif}} \geq 1$, we assume that every measurable $A \subseteq [a_n, b_n]$ satisfies $\mu_n(A) \leq \frac{c_{\text{unif}}}{w_n}\lambda(A)$. When $c_{\text{unif}} = 1$, this recovers the uniform distribution on $[a_n, b_n]$.

Theorem 6.2 (Probabilistic geometric stability of h-cover representatives). *Let $I \subseteq \mathbb{R}^2$ be finite, let $R \subseteq I$ satisfy $\alpha_{\text{Poly}}(R) = \alpha_{\text{Poly}}(I) = K$, and suppose R is an h-cover of I under Euclidean distance. Under the near-uniform threshold assumption above, for every $\delta \in (0,1]$,*

$$\Pr_{t\sim\mu_n}\left[\begin{array}{c}\text{Area}(\alpha_{\text{Poly}}(I \cap G(n,t))) - \text{Area}(\alpha_{\text{Poly}}(R \cap G(n,t))) \\ \leq P_K h + \pi h^2 + \dfrac{c_{\text{unif}} h D_K}{\delta}\end{array}\right] \geq 1-\delta,$$

where $D_K \triangleq \text{diam}(K)$ is the diameter of K and $P_K \triangleq \text{Perimeter}(K)$ is its perimeter.

Thus, in this two-dimensional instance, the abstract loss caused by replacing I with R after a linear guard decreases with the achieved covering radius h. In the notation of Definition 6.4, this instantiates the compatibility function as $\Omega_{I,\mathcal{G}}(h,\delta) \triangleq P_K h + \pi h^2 + \frac{c_{\text{unif}} h D_K}{\delta}$. This numerical instance shows that the framework of Section 6.2 can be realized by a concrete coverage metric and an explicit high-probability abstract-loss bound.

6.4 Posterior-Guided Representative Selection

The representative-selection framework in previous subsections is intentionally prior-driven. It chooses a compact set of concrete states before using information from a particular program execution, alarm, or posterior validation result. In this sense, the objective in Section 6.2 does not inspect the program paths that will later be analyzed; it only aims to improve coverage of the input set so that many possible future restrictions can be represented accurately. The two-dimensional numerical bound in Section 6.3, together with the experiment in Section 7.2, shows that this prior strategy can already provide accurate upper-bound estimates in our main numerical instance.

Additionally, posterior information can be used as a natural extension when the prior selection method is not sufficiently effective. This is because the compatibility requirement in Definition 6.4 should not be read as a universal property for all abstract domains or all program-induced restrictions: some abstractions may satisfy such a condition for natural choices of d_Σ and $\mathcal{G}_I$, while others may not. For instance, if the abstraction is highly discontinuous with respect to d_Σ, or if the relevant restrictions concentrate on regions poorly covered by the prior representatives, then an h-cover alone may not be the best use of a fixed representative budget. In such cases, posterior feedback can be used to reweight the concrete states from which future representatives are selected.

The posterior source can be a confirmed real alarm similar to Section 5.2. Suppose an assertion at location q has an error specification $\mathsf{Err} \in C = \mathcal{P}(\Sigma)$, describing the program states at q that violate the asserted property. For the path program $\Pi_{\text{entry}\rightsquigarrow q}$, the weakest precondition $\mathsf{wp}(\Pi_{\text{entry}\rightsquigarrow q}, \mathsf{Err})$ identifies states of the input set that may flow into the error region. A posterior-guided selector can

therefore increase the sampling weight of concrete states lying in, or near, this precondition. As a result, future upper-bound estimates devote more budget to states that matter for observed errors.

We leave this posterior-guided variant as an extension of the present upper-bound framework. The core development and evaluation in this paper focus on prior representative selection, while the posterior discussion above indicates how the same framework can be adapted when feedback from alarm validation is available as in Section 5.2. For a formalization and a small numerical example of this posterior upper-bound refinement, the reader can refer to the appendix.

7 Evaluation

This section evaluates the numerical instance of our framework by running it on multiple families of randomly generated input sets and programs. The goal is to test whether the proposed upper- and lower-bound methods are effective beyond the illustrative example.

Our upper and lower bounds provide different evidence on reported alarms: a small upper bound supports the possibility that an alarm is real, but a large upper bound may only reflect a loose over-estimate. Conversely, a large lower bound certifies unavoidable imprecision and supports the possibility of a false positive, while a small lower bound may simply mean that the lower-bound reasoning has not yet exposed the imprecision.

The experiments evaluate all three aspects of the numerical instance: prior representative selection for upper bounds, initial forward propagation for lower bounds, and posterior learning from false-positive feedback. For upper bounds, representative selection preserves over 91% of the concrete area at all assertion sites and removes more than 91% of the generator slack. For lower bounds, the initial forward logic already recovers 46.3% of the loss at the p_t program point on average, and posterior feedback raises this recovery to 64.4% when alarm-triage information is available. Since the prior representative method already targets the main source of looseness in generator-based upper propagation, our posterior experiment focuses on the lower endpoint, where the initial certificate is more conservative. Concretely, our experiments answer the following three questions:

RQ1. Can a small representative set produce tight upper-bound estimates at later assertion sites, compared with the generator baseline?

RQ2. Without posterior feedback, how informative are the initial lower-bound estimates produced by forward reasoning?

RQ3. Can posterior feedback effectively improve conservative lower-bound estimates?

We first describe our experimental setup in Section 7.1. Then, we answer RQ1, RQ2, and RQ3 in Section 7.2, Section 7.3, and Section 7.4, respectively.

7.1 Experimental Setup

We conducted all experiments on Linux machines with 2.6 GHz processors and 256 GB RAM.

Programs. Each program state is a point in $\mathbb{R}^2$. The benchmark program has the following control-flow shape:

$$\begin{aligned}&\texttt{affine}_1;\\ &\texttt{if } G_1 \texttt{ then affine}_2;\ (\texttt{if } G_2 \texttt{ then assert}_1 \texttt{ else } (\texttt{affine}_3;\ \texttt{assert}_2))\\ &\texttt{else affine}_4;\\ &\texttt{assert}_3.\end{aligned}$$

Here $\texttt{assert}_3$ is observed after the join of the G_1-then and G_1-else branches. Each affine command has the form $\mathbf{v} := Av + b$. The guards are fixed halfspaces, $G_1 = \{(x, y) \mid x \geq 0\}$ and $G_2 = \{(x, y) \mid y \geq 0\}$. We generate 100 random programs with a fixed random seed. All $\texttt{affine}_i$ are sampled

independently from the same distribution: each matrix entry of A is sampled uniformly from $[-1.5, 1.5]$, each translation entry of b from $[-2, 2]$, and matrices with $|\det A| < 0.05$ are rejected.

Abstract execution. Following the setting from Section 2, the ideal abstract domain is Poly, the domain of all convex polygons in $\mathbb{R}^2$. The implementation still computes in this domain, but after each affine, guard, or join transfer, a polygon with more than five edges is replaced by its smallest axis-aligned enclosing box. This creates the abstraction loss measured in the experiments.

Input sets. All input sets are finite point clouds inside the fixed equilateral triangle centered at the origin with side length $8\sqrt{3}$. We use six families of input-set distributions: uniform sampling, a centered truncated Gaussian, a three-component Gaussian mixture, an edge-biased Dirichlet distribution, a center-biased Dirichlet distribution, and a vertex-biased Dirichlet mixture. For each family we generate 100 input sets of 100,000 points using fixed random seeds.

Representative selection for upper-bound evaluation. Each input set is compressed to representatives with budget 800 using a fixed selection seed. The algorithm first retains all support points of $\alpha_{\mathsf{Poly}}(I)$. If the support points exceed the budget, they are all kept. Otherwise, the remaining budget is used to reduce the Euclidean covering radius $h_R = \max_{x \in I} \min_{r \in R} \|x - r\|_2$, so the selected set forms an empirical h_R-cover of the original point cloud. We evaluate three covering policies. The centroid-based policy triangulates the input hull from its polygon centroid, allocates the remaining budget across the triangles, and selects concrete points from the original point cloud inside each triangle. The square-grid and triangular-grid policies instead overlay the hull with square or triangular cells and choose concrete points from occupied cells, refining the cell size until the budget is used. In all cases, every original point is assigned to its nearest selected representative; the largest such assignment distance is the achieved h_R. Thus the selected representatives form an empirical h_R-cover, and the three policies differ only in how they spread the non-support representatives before this covering radius is measured. The selected set is reused across all 100 programs.

Generator-based baseline. For the upper-bound side, we compare our representative-based method with the generator-based baseline from prior EPL-style propagation [4]. This baseline keeps only the generator/support states of the initial abstract result and propagates those states through the program to estimate an upper loss. It is therefore a natural baseline for upper-bound estimation, but it does not provide a lower-bound certificate. The lower-bound experiments consequently evaluate our forward and posterior lower-bound method directly rather than comparing against a baseline.

Posterior feedback. For each program, $\texttt{assert}_1$ and $\texttt{assert}_2$ each receive 2000 halfspace constraints of the form $A\mathbf{v} \geq b$, where $A = (\cos\theta, \sin\theta)$. The angle θ is stratified over $[0, 2\pi)$; thresholds use a mixed stratification, with 60% sampled from $[-120, 120]$ and 40% from the central interval $[-30, 30]$. These constraints are generated once per program using fixed random seeds, and reused for every input set. A constraint is useful only when the abstract branch result violates it but all concrete points reaching that branch satisfy it. This means that the alarm raised by the abstract execution is a false-positive report. Following Section 5.2, the path program to $\texttt{assert}_1$ is $\texttt{affine}_1; G_1?; \texttt{affine}_2; G_2?$, while the one to $\texttt{assert}_2$ is $\texttt{affine}_1; G_1?; \texttt{affine}_2; (\neg G_2)?; \texttt{affine}_3$. Useful feedback is pulled back along these path programs, accumulated as posterior restrictions, and compared with the unchanged abstract result to update the lower bound.

Metrics. At each assertion site q, let A_q be the implemented abstract result, T_q the convex hull of all 100,000 concrete points reaching q, S_q the convex hull induced by the selected representatives, and G_q the convex hull induced only by the generator points. The true abstraction loss is $\mathrm{Area}(A_q) - \mathrm{Area}(T_q)$, and the selected upper loss is $\mathrm{Area}(A_q) - \mathrm{Area}(S_q)$. Since the abstract result A_q is fixed, the accuracy of this upper estimate under the area distance depends on how much of T_q's

Site	True loss	Rep. upper	Gen. upper	Rep. ret. (%)	Gen. ret. (%)	Rep. slack	Gen. slack	Slack red. (%)
$\texttt{assert}_1$	31.92	32.27	41.09	93.2	33.2	0.35	9.18	96.2
$\texttt{assert}_2$	32.73	33.04	42.03	91.3	33.8	0.32	9.30	96.6
$\texttt{assert}_3$	315.65	318.20	346.88	98.0	74.5	2.56	31.23	91.8

Table 1. Upper-bound accuracy, averaged over the three representative-selection policies. "Rep." denotes our representative-based method and "Gen." denotes the generator baseline. "ret." denotes the corresponding area retention percentage, while slack reduction ("Slack red.") is the percentage of generator slack removed by representative selection.

area is preserved by S_q. We therefore use area retention, $\mathrm{Area}(S_q)/\mathrm{Area}(T_q)$, as a primary metric. Additionally, the selected slack is the excess over the true loss: $(\mathrm{Area}(A_q) - \mathrm{Area}(S_q)) - (\mathrm{Area}(A_q) - \mathrm{Area}(T_q)) = \mathrm{Area}(T_q) - \mathrm{Area}(S_q)$. This is numerically the same as the area gap between the true and selected convex hulls, but it is interpreted as the looseness of the induced upper bound, so we use the term selected slack. We also report the same retention $\mathrm{Area}(G_q)/\mathrm{Area}(T_q)$ and slack $\mathrm{Area}(T_q) - \mathrm{Area}(G_q)$ for the generator baseline, and use the fraction of generator slack removed by selected representatives to quantify the improvement over the generator-based method.

For lower-bound evaluation, the metric recovery is the certified lower bound ℓ_q divided by the true loss $\mathrm{Area}(A_q) - \mathrm{Area}(T_q)$ at the program point being studied: $\mathrm{recovery}(q) = \ell_q/(\mathrm{Area}(A_q) - \mathrm{Area}(T_q))$. The main lower-bound target is the conservative estimate immediately after $\texttt{affine}_1; G_1?$. We also measure how the learned restrictions adjust the certified lower bound at $\texttt{assert}_3$.

7.2 Upper-Bound Results (RQ1)

The upper-bound experiment covers all three assertion sites. Table 1 summarizes the true loss, the upper bound using our approach, the generator-based upper bound, as well as the area retention and the selected slack for both methods.

Overall, generator-based propagation keeps only the initial support states, which is about 25 points on average. Our approach uses the budget of 800 points, still far below the 100,000 concrete points, but much more evenly cover the input set. This improves the area retained after later guards and affine maps: as shown in Table 1, retention for our approach is 93.2% at $\texttt{assert}_1$, 91.3% at $\texttt{assert}_2$, and 98.0% at $\texttt{assert}_3$ averaged over selection policies and input-set distributions. By contrast, the generator baseline retains only 33.2%, 33.8%, and 74.5% of the same concrete hull areas. The gap is especially large at $\texttt{assert}_1$ and $\texttt{assert}_2$, where guards filter away many states and hence generator points no longer cover the surviving region well. At $\texttt{assert}_3$, the outer if-then-else merge retains all the program states, so generator retention is already higher; nevertheless, representatives still improve retention from 74.5% to 98.0%. This shows that the representative-based method remains effective even in the case where the program largely preserves states.

Table 1 and fig. 9 also show that the representative-based estimate nearly recovers the true loss. The remaining selected slack is only 0.35, 0.32, and 2.56 area units at the three assertion sites, compared with generator slack 9.18, 9.30, and 31.23. Equivalently, the selected representatives remove 96.2%, 96.6%, and 91.8% of the generator slack. In the arrow plot, this appears as a short residual distance from the true-loss point to the representative upper bound, and a much longer distance to the generator upper bound.

In conclusion, the upper-bound experiment confirms that our representative-state selection is effective for upper-bound estimation. With a small budget relative to the original concrete

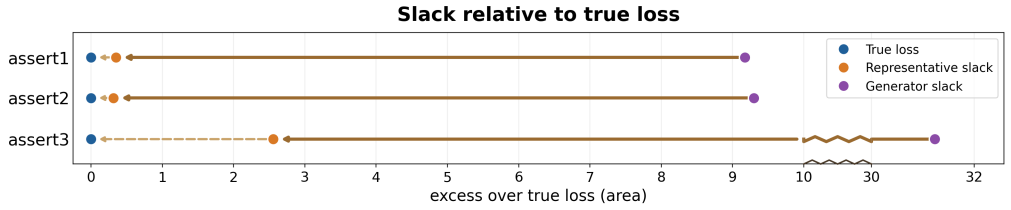


Fig. 9. Upper-bound slack relative to the true loss. The blue point marks the true loss at zero excess, the orange point marks the representative-based slack, and the purple point marks the generator-based slack.

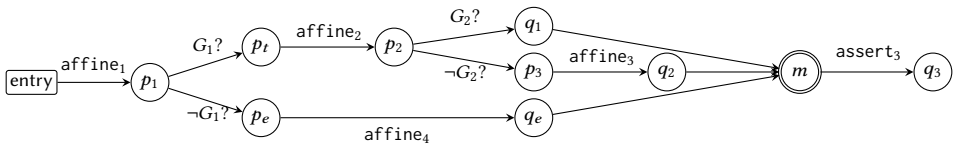


Fig. 10. CFG of the lower-bound benchmark program used in the experiments. The posterior-learning experiment uses feedback at q_1 and q_2 to improve the conservative lower bound at p_t, and then propagates the learned posterior information toward q_3.

Location	True loss	Initial LB	Initial rec.	Post. LB	Post. rec.	q_1-post. rec.	q_2-post. rec.
p_t	73.04 / 56.81	44.28 / 26.41	0.463 / 0.477	53.69 / 34.98	0.644 / 0.711	0.528 / 0.555	0.531 / 0.577
q_3	315.65 / 220.69	0.00 / 0.00	0.000 / 0.000	93.70 / 20.99	0.228 / 0.129	0.145 / 0.001	0.159 / 0.015

Table 2. Lower-bound estimates before and after posterior feedback. Each cell reports mean / median. Here "rec." denotes recovery ratio and "LB" denotes lower bound. The initial columns answer RQ2, while the posterior columns summarize the learning effect studied in RQ3.

point cloud, our method preserves much more of the concrete hull after program execution and substantially tightens the upper bound that would be obtained from generator-only propagation.

7.3 Initial Lower-Bound Results (RQ2)

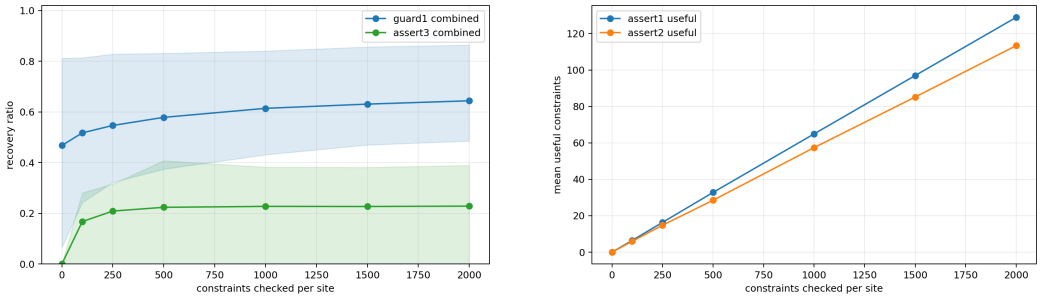
We refer to the program locations using the CFG as shown in Figure 10: p_t is the point immediately after taking the G_1-then branch, and q_1, q_2, q_3 are the three assertion sites.

The columns in Table 2 show that the initial estimates of lower bounds are informative but conservative. At p_t, the forward logic already certifies a substantial part of the loss, with mean recovery 46.3% and median recovery 47.7%. At q_3, however, the outer join makes the initial lower bound completely uninformative, while the true loss is 315.65 on average.

This matches the conservative behavior of the forward lower-bound logic: it must remain sound without knowing which part of the abstract error survives later guards and joins. This conservatism is most visible on longer paths with several filtering and join points, although shorter fragments can still yield informative lower bounds as in p_t.

7.4 Posterior Lower-Bound Learning (RQ3)

After the initial forward estimation of lower bounds, false-positive feedback from the user can further improve them. We checked 2000 constraints at q_1 and q_2 respectively. Among the assertions that trigger abstract alarms, we learn only from the useful (false-positive) ones, namely those



(a) Recovery vs. checked constraints

(b) Useful constraints

Fig. 11. Lower-bound posterior learning. More checked constraints produce more useful feedback, and the learned restrictions improve the lower bound at p_t and also propagate to q_3.

satisfied by all concrete states reaching the site. This feedback is pulled back along the corresponding path in Figure 10, combined at p_t, and then propagated forward toward q_3.

Figure 11 shows that the useful feedback is sparse but effective. We checked 4000 constraints in total for each input-set–program pair, with mean useful count 242.3 and median useful count 68.0. The useful fraction is limited because the constraints are fixed for all input sets of the same program: many either do not trigger an abstract alarm, or they are real violations by the concrete points rather than false positives. However, Table 2 shows that the learned restrictions raise recovery at p_t from 46.3% to 64.4% on average and from 47.7% to 71.1% at the median. The same posterior information also propagates to q_3, where recovery rises from zero to mean 22.8%.

Figure 11(a) further shows the learning dynamics. Recovery increases steadily as more constraints are checked, with the largest gains in the earlier part of the curve and a gradual flattening as the easily discoverable false positives are exhausted. The propagated recovery at q_3 follows the same trend but remains weaker because q_3 also includes the outer-else branch and has a much larger true loss: the mean true loss at p_t is 73.0, while the mean true loss at q_3 is 315.6.

In conclusion, posterior feedback provides a useful complement to the initial forward lower-bound calculation. It recovers additional certified loss from false-positive alarms and substantially tightens the lower endpoint at both the learned program point and its downstream assertion.

8 Related Work

Completeness and partial completeness. Completeness is a classical notion in abstract interpretation for characterizing when abstraction introduces no loss of precision [1, 2, 11]. Global completeness requires exact preservation for all input sets, while local completeness restricts this requirement to a particular input set. Partial completeness further relaxes this binary view by allowing a bounded precision loss measured by the abstract-domain distance [3]. Our work follows this quantitative perspective: instead of asking only whether an analysis is complete, we reason about how much imprecision is introduced.

Error propagation logic. The closest technical predecessor is the error propagation logic (EPL) of Campion et al. [4], which derives upper bounds on abstract imprecision compositionally and uses generators to reduce the propagation cost. We build on this distance-based framework, but extend it in two directions. First, we reason about both upper and lower bounds, so large upper bounds can be distinguished from unavoidable loss. Second, for upper-bound reasoning, we replace

the fixed generator choice with configurable representative states that preserve more concrete information for later guards and joins.

Probabilistic confidence. A recent framework for abstract interpretation with confidence defines alarm confidence as the probability that an abstract execution is locally complete over a distribution of matching concrete programs [20]. Our work is complementary. Rather than assigning probabilities to local-completeness events, we quantify distance-based imprecision for a fixed program and input set. At the same time, the probabilistic event in that framework could be relaxed from local completeness to partial completeness, as also suggested by its limitations; our two-sided bounds provide a way to reason about such relaxed precision events.

9 Conclusion

We presented a two-sided framework for quantifying imprecision in abstract interpretation by deriving both upper and lower bounds on abstract-domain distance. The framework extends partial completeness from a one-sided estimate to an interval view of precision loss, refines upper-bound reasoning through representative concrete states, and strengthens lower-bound reasoning through posterior learning from validated feedback. Additionally, our numerical instantiation and evaluations show that the framework works effectively in such a concrete setting.

References

[1] Roberto Bruni, Roberto Giacobazzi, Roberta Gori, Isabel Garcia-Contreras, and Dusko Pavlovic. 2019. Abstract extensionality: on the properties of incomplete abstract interpretations. *Proceedings of the ACM on Programming Languages* 4, POPL (2019), 1–28.

[2] Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. 2021. A logic for locally complete abstract interpretations. In *Proceedings of the 36th Annual ACM/IEEE Symposium on Logic in Computer Science* (Rome, Italy) *(LICS '21)*. Association for Computing Machinery, New York, NY, USA, Article 52, 13 pages. doi:10.1109/LICS52264.2021.9470608

[3] Marco Campion, Mila Dalla Preda, and Roberto Giacobazzi. 2022. Partial (In)Completeness in abstract interpretation: limiting the imprecision in program analysis. *Proc. ACM Program. Lang.* 6, POPL, Article 59 (Jan. 2022), 31 pages. doi:10.1145/3498721

[4] Marco Campion, Mila Dalla Preda, Roberto Giacobazzi, and Caterina Urban. 2026. A Logic for the Imprecision of Abstract Interpretations. *Proc. ACM Program. Lang.* 10, POPL, Article 65 (Jan. 2026), 29 pages. doi:10.1145/3776707

[5] Patrick Cousot. 2021. *Principles of abstract interpretation.* MIT Press.

[6] Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (Los Angeles, California) *(POPL '77)*. Association for Computing Machinery, New York, NY, USA, 238–252. doi:10.1145/512950.512973

[7] Patrick Cousot and Radhia Cousot. 1977. Static determination of dynamic properties of generalized type unions. *ACM SIGOPS Operating Systems Review* 11, 2 (1977), 77–94.

[8] Patrick Cousot and Radhia Cousot. 1979. Systematic design of program analysis frameworks. In *Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (San Antonio, Texas) *(POPL '79)*. Association for Computing Machinery, New York, NY, USA, 269–282. doi:10.1145/567752.567778

[9] Patrick Cousot and Radhia Cousot. 1992. Abstract Interpretation Frameworks. *J. Log. Comput.* 2 (1992), 511–547. https://api.semanticscholar.org/CorpusID:6384543

[10] Dino Distefano, Manuel Fähndrich, Francesco Logozzo, and Peter W. O'Hearn. 2019. Scaling static analyses at Facebook. *Commun. ACM* 62, 8 (July 2019), 62–70. doi:10.1145/3338112

[11] Roberto Giacobazzi, Francesco Logozzo, and Francesco Ranzato. 2015. Analyzing program analyses. *ACM SIGPLAN Notices* 50, 1 (2015), 261–273.

[12] Dexter Kozen. 1997. Kleene algebra with tests. *ACM Trans. Program. Lang. Syst.* 19, 3 (May 1997), 427–443. doi:10.1145/256167.256195

[13] Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. 1999. *Principles of Program Analysis.* Springer. doi:10.1007/978-3-662-03811-6

[14] Peter W O'Hearn. 2018. Continuous reasoning: Scaling the impact of formal methods. In *Proceedings of the 33rd annual ACM/IEEE symposium on logic in computer science*. 13–25.

[15] Peter W O'Hearn. 2019. Incorrectness logic. *Proceedings of the ACM on Programming Languages* 4, POPL (2019), 1–32.
[16] Henry Gordon Rice. 1953. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical society* 74, 2 (1953), 358–366.
[17] Xavier Rival and Kwangkeun Yi. 2020. *Introduction to static analysis: an abstract interpretation perspective.* Mit Press.
[18] Caitlin Sadowski, Edward Aftandilian, Alex Eagle, Liam Miller-Cushon, and Ciera Jaspan. 2018. Lessons from building static analysis tools at google. *Commun. ACM* 61, 4 (2018), 58–66.
[19] Ravi Sethi. 1983. Control Flow Aspects of Semantics-Directed Compiling. *ACM Transactions on Programming Languages and Systems* 5, 4 (1983), 554–595. doi:10.1145/69575.357227
[20] Yuanfeng Shi, Ziyue Jin, and Xin Zhang. 2026. Abstract Interpretation with Confidence: Quantifying the Precision of Dataflow Analysis with Probabilities. *Proc. ACM Program. Lang.* 10, PLDI, Article 273 (June 2026), 25 pages. doi:10.1145/3808351
[21] Robert Endre Tarjan. 1981. Fast Algorithms for Solving Path Problems. *J. ACM* 28, 3 (July 1981), 594–614. doi:10.1145/322261.322273
[22] Robert Endre Tarjan. 1981. A Unified Approach to Path Problems. *J. ACM* 28, 3 (July 1981), 577–593. doi:10.1145/322261.322272
[23] Glynn Winskel. 1993. *The formal semantics of programming languages: an introduction.* MIT press.
[24] Xin Zhang, M. Naik, and Hongseok Yang. 2013. Finding optimum abstractions in parametric dataflow analysis. *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (2013). https://api.semanticscholar.org/CorpusID:14523587

A Proofs for the Lower-Bound Soundness Theorems

This section gives the proof material for the four lower-bound results in Section 5. The proofs use the standard soundness and monotonicity assumptions for the concrete and abstract transfer functions stated in the main text, together with the chain-order property of the abstract-domain pre-metric.

A.1 Proof of Theorem 5.1

We prove a stronger statement for the internal pair judgment. Suppose $\langle a, l\rangle \overset{\mathsf{P}}{\underset{L,\mathcal{A}}{\Rightarrow}} \langle a', l'\rangle$ is derivable. Then for every $\hat{a} \in \mathrm{Wit}_L(a, l)$, if $\hat{a}_{\mathsf{P}} \triangleq \alpha([\![\mathsf{P}]\!]_C(\gamma(\hat{a})))$, then $\hat{a}_{\mathsf{P}} \sqsubseteq_{\mathcal{A}} a'$ and $\delta_{\mathcal{A}}(\hat{a}_{\mathsf{P}}, a') \geq l'$. The first part is ordinary soundness of abstract interpretation; the second part is proved by induction on the derivation.

For **Base(L)**, the result is exactly the definition of $\textsc{LTrans}_{\mathcal{A}}$. For **Weaken(L)**, the induction hypothesis gives $\delta_{\mathcal{A}}(\hat{a}_{\mathsf{P}}, a') \geq l'$, and the premise $l' \geq \hat{l}$ gives the weakened certificate.

For **Seq(L)**, let $\hat{a}_1 = \alpha([\![\mathsf{P}_1]\!]_C(\gamma(\hat{a})))$. The induction hypothesis for P_1 gives $\hat{a}_1 \sqsubseteq_{\mathcal{A}} a_1$ and $\delta_{\mathcal{A}}(\hat{a}_1, a_1) \geq l_1$, so $\hat{a}_1 \in \mathrm{Wit}_L(a_1, l_1)$. Applying the induction hypothesis to P_2 from this witness gives the required bound for $\mathsf{P}_1; \mathsf{P}_2$.

For **Join(L)**, apply the induction hypothesis to both branches. The exact abstract branch results $\hat{a}_1$ and $\hat{a}_2$ therefore belong to $\mathrm{Wit}_L(a_1, l_1)$ and $\mathrm{Wit}_L(a_2, l_2)$, respectively. The exact abstract result of nondeterministic choice is $\hat{a}_1 \vee_{\mathcal{A}} \hat{a}_2$, while the implemented abstract result is $a_1 \vee_{\mathcal{A}} a_2$. The premise $\textsc{LJoin}_{\mathcal{A}}$ lower-bounds the distance for all such witnesses, hence also for the actual pair of branch witnesses.

For **Iterate(L)**, the premise $\textsc{LimIter}_{\mathcal{A}}$ states precisely that the proposed sequence is a valid lower-bound sequence against the final abstract loop invariant a_∞, and that its limiting certificate is at least $\bar{l}$. Thus the strengthened property holds for P^*.

Finally consider **Program(L)**. Since $\alpha(I) \in \mathrm{Wit}_L(\alpha(I), 0)$, the strengthened property gives a lower bound for running P from $\gamma(\alpha(I))$. By monotonicity, $\alpha([\![\mathsf{P}]\!]_C(I)) \sqsubseteq_{\mathcal{A}} \alpha([\![\mathsf{P}]\!]_C(\gamma(\alpha(I)))) \sqsubseteq_{\mathcal{A}} [\![\mathsf{P}]\!]_{\mathcal{A}}(\alpha(I))$. The chain-order property of $\delta_{\mathcal{A}}$ can only increase the distance when we move the first argument downward in this chain. Hence the same lower bound holds for $\delta_{\mathcal{A}}(\alpha([\![\mathsf{P}]\!]_C(I)), [\![\mathsf{P}]\!]_{\mathcal{A}}(\alpha(I)))$, which is $\mathrm{err}_{\mathcal{A}}(\mathsf{P}, I)$.

A.2 Proof of Theorem 5.2

Let $a_I \in \mathrm{Wit}_L(a, l)$, and define $c_n \triangleq \bigvee_C \{F_C^i(\gamma(a_I)) \mid 0 \leq i \leq n\}$. We show by induction on n that $\delta_{\mathcal{A}}(\alpha(c_n), a_\infty) \geq \lambda_n$ for every such witness a_I. Since $a_\infty = [\![\mathsf{P}^*]\!]_{\mathcal{A}}(a)$, ordinary abstract soundness also ensures $\alpha(c_n) \sqsubseteq_{\mathcal{A}} a_\infty$ at every finite iteration.

For $n = 0$, we have $c_0 = \gamma(a_I)$. Since $\alpha \circ \gamma(a_I) \leq_{\mathcal{A}} a_I$, using the chain-order property of $\delta_{\mathcal{A}}$ and the initial condition $\delta_{\mathcal{A}}(a_I, a_\infty) \geq \lambda_0$ we have $\delta_{\mathcal{A}}(\alpha(c_0), a_\infty) \geq \lambda_0$.

For the induction step, assume $\delta_{\mathcal{A}}(\alpha(c_n), a_\infty) \geq \lambda_n$. Together with $\alpha(c_n) \sqsubseteq_{\mathcal{A}} a_\infty$, this means $\alpha(c_n) \in \mathrm{Wit}_L(a_\infty, \lambda_n)$. Instantiate the induction condition with $\hat{a} = \alpha(c_n)$. By $\vee_C$-distributivity of F_C, $c_{n+1} = \gamma(a_I) \vee_C F_C(c_n)$. Moreover, $c_n \subseteq \gamma(\hat{a})$, so monotonicity of F_C gives $c_{n+1} \subseteq \gamma(a_I) \vee_C F_C(\gamma(\hat{a}))$. The induction condition states that the abstraction of the larger right-hand side is at distance at least λ_{n+1} from a_∞. Since $\alpha(c_{n+1})$ lies below that abstraction and both lie below a_∞, the chain-order property of $\delta_{\mathcal{A}}$ yields $\delta_{\mathcal{A}}(\alpha(c_{n+1}), a_\infty) \geq \lambda_{n+1}$.

Thus the finite-iteration distance condition in $\textsc{LimIter}_{\mathcal{A}}$ holds for every input witness. The theorem's assumptions also include $a_\infty = [\![\mathsf{P}^*]\!]_{\mathcal{A}}(a)$ and $\liminf_{n\to\infty} \lambda_n \geq \bar{l}$. Therefore all clauses of $\textsc{LimIter}_{\mathcal{A}}(a, l, F_C, F_{\mathcal{A}}, (\lambda_n)_{n\in\mathbb{N}}, a_\infty, \bar{l})$ hold.

A.3 Proof of Theorem 5.3

Fix the input set I. Write $C_r(I)$ for the exact concrete result reaching location r. We show that every step of Algorithm 1 preserves $\alpha(C_r(I)) \sqsubseteq_{\mathcal{A}} w_r \sqsubseteq_{\mathcal{A}} a_r$ and $l_r \le \delta_{\mathcal{A}}(\alpha(C_r(I)), a_r)$.

First consider the backward annotations. We claim that if $(r, S) \in \mathsf{Bwd}(\Pi_{\mathsf{entry}\rightsquigarrow q}, R)$ and $\textsc{FalseAlarm}(q, R, I)$ holds, then $C_r(I) \subseteq S$. This follows by structural induction on the path program. For an atomic located edge $e^{p\to r}$, the annotation is (r, R), and the claim is exactly the validated postcondition at the assertion end of the considered path fragment. For a sequence $\Pi_1;\Pi_2$, the annotations from Π_2 are kept, and the annotations for Π_1 are computed with $\mathrm{wp}(\Pi_2, R)$. The defining property of weakest preconditions ensures that any concrete state satisfying this precondition reaches a state satisfying R after Π_2. For a nondeterministic choice, the same postcondition must hold on each branch, so the union of branch annotations is sound.

Thus, for every generated restriction $S_r^{q,R}$, the reachable concrete states at r satisfy $C_r(I) \subseteq S_r^{q,R}$, and therefore $\alpha(C_r(I)) \sqsubseteq_{\mathcal{A}} \alpha(S_r^{q,R})$. Before the update, the invariant gives $\alpha(C_r(I)) \sqsubseteq_{\mathcal{A}} w_r$. Hence after the backward meet $w_r \leftarrow w_r \wedge_{\mathcal{A}} \alpha(S_r^{q,R})$, the updated value still over-approximates $\alpha(C_r(I))$, and it remains below the old w_r, hence below a_r.

Now consider the forward propagation pass. For a non-entry location r, every reachable concrete state at r arrives through some incoming edge (p, e, r). By the induction invariant at predecessors, $\alpha(C_p(I)) \sqsubseteq_{\mathcal{A}} w_p$. Soundness and monotonicity of the edge transformer give $\alpha(\llbracket e \rrbracket_C(C_p(I))) \sqsubseteq_{\mathcal{A}} \llbracket e \rrbracket_{\mathcal{A}}(w_p)$. Joining over all incoming edges yields $\alpha(C_r(I)) \sqsubseteq_{\mathcal{A}} \widehat{w}_r$. Since the invariant also gives $\alpha(C_r(I)) \sqsubseteq_{\mathcal{A}} w_r$, the update $w_r \leftarrow w_r \wedge_{\mathcal{A}} \widehat{w}_r$ preserves $\alpha(C_r(I)) \sqsubseteq_{\mathcal{A}} w_r \sqsubseteq_{\mathcal{A}} a_r$. A single topological pass suffices because the CFG is acyclic after loop bodies are collapsed into summary edges.

Finally, the lower-bound update sets $l_r \leftarrow \max\{l_r, \delta_{\mathcal{A}}(w_r, a_r)\}$. The old l_r is sound by the invariant. For the new candidate, we have $\alpha(C_r(I)) \sqsubseteq_{\mathcal{A}} w_r \sqsubseteq_{\mathcal{A}} a_r$, so the chain-order property implies $\delta_{\mathcal{A}}(w_r, a_r) \le \delta_{\mathcal{A}}(\alpha(C_r(I)), a_r)$. Therefore the maximum is still a sound lower bound. This proves that $\mathsf{Inv}_I(r)$ holds at every location after the update.

A.4 Proof of Theorem 5.4

It suffices, by Theorem 5.1, to show that the numerical rules in Figure 8 are sound instances of $\textsc{LTrans}_{\mathsf{Poly}}$, and that the implementation's box simplification contributes a sound base loss.

For the simplification step, let K be the best convex-polygon result before complexity control and let $\rho(K)$ be the implemented result. By construction $K \subseteq \rho(K)$, and the exact result represented before simplification is still contained in K. The area introduced by this implementation step is therefore $\delta_{\mathsf{Poly}}(K, \rho(K)) = \mathrm{Vol}(\rho(K)) - \mathrm{Vol}(K)$, which is a sound lower bound on the imprecision caused by replacing K with $\rho(K)$.

For an affine command $\boldsymbol{v} := Av + b$, take any witness $\widehat{K} \in \mathrm{Wit}_L(K, l)$. Then $\widehat{K} \subseteq K$ and $\mathrm{Vol}(K) - \mathrm{Vol}(\widehat{K}) \ge l$. Affine images preserve inclusion, and planar area is scaled by $|\det A|$. Hence, writing $T(u) = Au + b$,

$$\begin{aligned}\delta_{\mathsf{Poly}}(T(\widehat{K}), T(K)) &= \mathrm{Vol}(T(K)) - \mathrm{Vol}(T(\widehat{K})) \\ &= |\det A|\big(\mathrm{Vol}(K) - \mathrm{Vol}(\widehat{K})\big) \\ &\ge |\det A|\, l.\end{aligned}$$

Thus the affine rule satisfies $\textsc{LTrans}_{\mathsf{Poly}}$.

For a guard $G?$, let $K' = \mathrm{Conv}(K \cap G)$ and $\widehat{K}' = \mathrm{Conv}(\widehat{K} \cap G)$. Since $\widehat{K} \subseteq K$, we have $\widehat{K}' \subseteq K'$. Also $\widehat{K}' \subseteq \widehat{K}$, because $\widehat{K}$ is convex. Therefore

$$\begin{aligned}\delta_{\mathrm{Poly}}(\widehat{K}', K') &= \mathrm{Vol}(K') - \mathrm{Vol}(\widehat{K}') \\ &= \big(\mathrm{Vol}(K) - \mathrm{Vol}(\widehat{K}')\big) - \big(\mathrm{Vol}(K) - \mathrm{Vol}(K')\big) \\ &\geq \big(\mathrm{Vol}(K) - \mathrm{Vol}(\widehat{K})\big) - \Delta_G(K) \\ &\geq l - \Delta_G(K).\end{aligned}$$

The distance is nonnegative, so it is at least $\max\{0, l - \Delta_G(K)\}$. Hence the guard rule is also a sound $\textsc{LTrans}_{\mathrm{Poly}}$ instance.

All derivations using these numerical transfer rules are therefore derivations of the general forward logic, and the claimed lower bound follows from Theorem 5.1.

B Proof Material for the Upper-Bound Instantiation

This appendix records the proofs for the monotonicity theorem Theorem 6.1 in Section 6.1 and for the two-dimensional numerical upper-bound theorem in Section 6.3.

B.1 Proof of Theorem 6.1

Theorem B.1 (Soundness and monotonicity). *For every $R \subseteq I$, $\mathrm{err}_{\mathcal{A}}(\mathsf{P}, I) \leq U_R(\mathsf{P}, I)$. Moreover, if $R_1 \subseteq R_2 \subseteq I$, then $U_{R_2}(\mathsf{P}, I) \leq U_{R_1}(\mathsf{P}, I)$.*

Proof. Monotonicity of the concrete semantics and α, together with soundness of the abstract semantics, gives

$$\alpha(\llbracket \mathsf{P} \rrbracket_C(R)) \leq_{\mathcal{A}} \alpha(\llbracket \mathsf{P} \rrbracket_C(I)) \leq_{\mathcal{A}} \llbracket \mathsf{P} \rrbracket_{\mathcal{A}}(\alpha(I)).$$

The first result follows from chain-order compatibility of $\delta_{\mathcal{A}}$. For $R_1 \subseteq R_2$, inserting $\alpha(\llbracket \mathsf{P} \rrbracket_C(R_2))$ into the same chain yields the second. □

B.2 Deterministic Geometric Covering Bound

The following lemma is the deterministic geometric fact used inside the proof of Theorem 6.2.

Lemma B.2 (Deterministic geometric covering bound). *Let $I \subseteq \mathbb{R}^2$ be finite, let $R \subseteq I$ be an h-cover under Euclidean distance, and let $G(n,t) = \{x \in R^2 \mid n^\top x \geq t\}$ with $\|n\|_2 = 1$. Define*

$$G^{+h}(n,t) \triangleq G(n, t+h)$$

and

$$B_I(n,t,h) \triangleq \mathrm{Area}(\mathrm{Conv}(I \cap G(n,t))) - \mathrm{Area}(\mathrm{Conv}(I \cap G^{+h}(n,t))).$$

Then

$$\begin{aligned}&\mathrm{Area}(\mathrm{Conv}(I \cap G(n,t))) - \mathrm{Area}(\mathrm{Conv}(R \cap G(n,t))) \\ &\quad \leq B_I(n,t,h) + \mathrm{Perimeter}(\mathrm{Conv}(I \cap G(n,t)))h + \pi h^2.\end{aligned}$$

In particular, if $K = \alpha_{\mathrm{Poly}}(I)$, then the right-hand side is at most $B_I(n,t,h) + \mathrm{Perimeter}(K)h + \pi h^2$.

Proof. For readability, write G for $G(n,t)$ and G^{+h} for $G^{+h}(n,t)$, and define

$$Q = \mathrm{Conv}(I \cap G), \qquad Q^{+h} = \mathrm{Conv}(I \cap G^{+h}), \qquad P = \mathrm{Conv}(R \cap G).$$

Since $R \subseteq I$, we have $P \subseteq Q$. Our goal is to upper-bound $\mathrm{Area}(Q) - \mathrm{Area}(P)$.

Step 1: surviving representatives cover the inward-shifted points. We first show that

$$Q^{+h} \subseteq P \oplus B_h, \qquad B_h = \{z \in R^2 \mid \|z\|_2 \le h\}.$$

Take any $x \in I \cap G^{+h}$. Since R is an h-cover of I, there exists $r \in R$ such that $\|x - r\|_2 \le h$. Moreover, $n^\top x \ge t + h$, so

$$n^\top r = n^\top x + n^\top (r - x) \ge n^\top x - \|n\|_2 \|r - x\|_2 \ge t.$$

Thus $r \in R \cap G$. Now take any $z \in Q^{+h}$. Write $z = \sum_i \lambda_i x_i$ with $x_i \in I \cap G^{+h}$, $\lambda_i \ge 0$, and $\sum_i \lambda_i = 1$. Choose $r_i \in R \cap G$ such that $\|x_i - r_i\|_2 \le h$, and let $r = \sum_i \lambda_i r_i$. Then $r \in P$ and

$$\|z - r\|_2 \le \sum_i \lambda_i \|x_i - r_i\|_2 \le h.$$

Therefore $z \in P \oplus B_h$, proving $Q^{+h} \subseteq P \oplus B_h$.

Step 2: decomposing the area loss. Because $Q^{+h} \subseteq Q$,

$$Q \setminus P \subseteq (Q \setminus Q^{+h}) \cup (Q^{+h} \setminus P).$$

Hence

$$\mathrm{Area}(Q) - \mathrm{Area}(P) \le \mathrm{Area}(Q) - \mathrm{Area}(Q^{+h}) + \mathrm{Area}(Q^{+h} \setminus P).$$

The first term is exactly $B_I(n, t, h)$.

Step 3: bounding the covering error. From Step 1,

$$Q^{+h} \setminus P \subseteq (P \oplus B_h) \setminus P.$$

By the Steiner formula for planar convex bodies,

$$\mathrm{Area}(P \oplus B_h) - \mathrm{Area}(P) = \mathrm{Perimeter}(P)h + \pi h^2.$$

Since $P \subseteq Q$ and both are planar convex bodies, monotonicity of perimeter gives $\mathrm{Perimeter}(P) \le \mathrm{Perimeter}(Q)$. Therefore

$$\mathrm{Area}(Q^{+h} \setminus P) \le \mathrm{Perimeter}(Q)h + \pi h^2.$$

Combining the bounds yields the stated inequality. The final perimeter bound follows from $Q = \mathrm{Conv}(I \cap G(n, t)) \subseteq K$ and perimeter monotonicity. □

B.3 Probabilistic Boundary Loss

We now record the probabilistic part used to derive Theorem 6.2. The near-uniform threshold assumption is stated in Section 6.3; here we prove the boundary-loss estimate it implies.

The discrete boundary loss induced by moving the guard boundary inward by h is

$$B_I(n, t, h) = \mathrm{Area}(\mathrm{Conv}(I \cap G(n, t))) - \mathrm{Area}(\mathrm{Conv}(I \cap G^{+h}(n, t))).$$

Equivalently, if $A_n(t) = \mathrm{Area}(\mathrm{Conv}(I \cap G(n, t)))$, then $B_I(n, t, h) = A_n(t) - A_n(t + h)$.

Lemma B.3 (Integral bound for the discrete boundary loss). *For every fixed unit direction $n \in \mathbb{S}^1$ and every $h > 0$,*

$$\int_{\mathbb{R}} B_I(n, t, h)\, dt = h\ \mathrm{Area}(\mathrm{Conv}(I)).$$

Proof. Since I is finite, $A_n(t)$ is a bounded non-increasing step function of t. It can only change when t crosses one of the projected values $n^\top x$ for $x \in I$. Let $s_1, \dots, s_q$ be the jump points, and let $\Delta_k = A_n(s_k^-) - A_n(s_k^+) \geq 0$ be the decrease at s_k. Then

$$A_n(t) - A_n(t+h) = \sum_{k=1}^{q} \Delta_k \, 1[s_k - h \leq t < s_k].$$

Integrating gives

$$\int_{\mathbb{R}} B_I(n,t,h)\, dt = h \sum_{k=1}^{q} \Delta_k.$$

The sum of all decreases is $A_n(-\infty) - A_n(+\infty) = \mathrm{Area}(\mathrm{Conv}(I))$, so the result follows. □

Lemma B.4 (Expected boundary loss under near-uniform thresholds). *Under the near-uniform threshold assumption, for every fixed direction $n \in \mathbb{S}^1$,*

$$\mathbb{E}_{t\sim\mu_n}[B_I(n,t,h)] \leq c_{\mathrm{unif}} h \frac{\mathrm{Area}(\mathrm{Conv}(I))}{w_n}.$$

Proof. By the near-uniform threshold assumption, μ_n is dominated by $(c_{\mathrm{unif}}/w_n)\lambda$ on $[a_n, b_n]$. Hence

$$\begin{aligned}
\mathbb{E}_{t\sim\mu_n}[B_I(n,t,h)] &= \int_{[a_n,b_n]} B_I(n,t,h)\, d\mu_n(t) \\
&\leq \frac{c_{\mathrm{unif}}}{w_n} \int_{a_n}^{b_n} B_I(n,t,h)\, dt \\
&\leq \frac{c_{\mathrm{unif}}}{w_n} \int_{\mathbb{R}} B_I(n,t,h)\, dt.
\end{aligned}$$

The claim follows from Lemma B.3. □

Corollary B.5 (Diameter-level probabilistic bound). *Under the near-uniform threshold assumption, for every fixed direction $n \in \mathbb{S}^1$,*

$$\mathbb{E}_{t\sim\mu_n}[B_I(n,t,h)] \leq c_{\mathrm{unif}} h D_K,$$

where $D_K = \mathrm{diam}(\mathrm{Conv}(I))$. Consequently, for every $\eta > 0$,

$$\Pr_{t\sim\mu_n}[B_I(n,t,h) > \eta] \leq \frac{c_{\mathrm{unif}} h D_K}{\eta}.$$

Proof. The average slice length $\mathrm{Area}(\mathrm{Conv}(I))/w_n$ is bounded by $D_K = \mathrm{diam}(\mathrm{Conv}(I))$. Combining this observation with Lemma B.4 gives the expectation bound. Since $B_I(n,t,h) \geq 0$, Markov's inequality gives the probability bound. □

Taking $\eta = c_{\mathrm{unif}} h D_K/\delta$ in Corollary B.5 and combining it with Lemma B.2 yields Theorem 6.2.

C Loop Lower-Bound Validation Detail

It is important that the lower-bound sequence is compared with the final invariant a_∞, not with the moving abstract iterate $a_n = \bigvee_{\mathcal{A}} \{F^i_{\mathcal{A}}(\alpha(I)) \mid 0 \leq i \leq n\}$. The synchronous distance between $\alpha(I_n)$ and a_n can be misleading because the abstract side is still moving.

Example. Consider the interval abstraction, input set $I = \{1, 3\}$, and

$$\mathtt{P} \equiv \mathtt{if}\ x = 2\ \mathtt{then}\ x := x + 4\ \mathtt{else}\ x := x + 2.$$

The concrete semantics never takes the true branch from this input set:

$$I = \{1, 3\}, \qquad F_C(I) = \{3, 5\}, \qquad F_C^2(I) = \{5, 7\}, \dots$$

so the concrete loop prefixes satisfy $\alpha(I_n) = [1, 2n+3]$ for $n \geq 1$, and the final concrete loop abstraction is $[1, \infty]$. On the abstract side, however, the interval $[1, 3]$ may contain 2, so the abstract semantics must keep both branches:

$$F_{\mathcal{A}}([1,3]) = [3,6], \qquad [1,3] \vee_{\mathcal{A}} F_{\mathcal{A}}([1,3]) = [1,6].$$

Continuing the abstract iteration gives $a_n = [1, 2n+4]$ for $n \geq 1$, while $a_\infty = [1, \infty]$. Hence the final loop imprecision is $\delta_{\mathcal{A}}([1,\infty],[1,\infty]) = 0$, but the synchronous distance between $\alpha(I_n) = [1, 2n+3]$ and $a_n = [1, 2n+4]$ remains nonzero. This is why LimIter validates candidate lower-bound sequences against a_∞ rather than against a_n.

D Posterior Upper-Bound Refinement

Section 6 selects representatives before later restrictions are known. This gives broad coverage, but a fixed budget can remain loose for a particular restriction that is eventually encountered. A real alarm provides posterior information because some concrete input reaches the violating region. We use this information only to add input representatives and recompute the representative-based upper bound. The original abstract analysis result is unchanged.

Weakest possible preconditions. For a real alarm at $\texttt{assert}_q(S)$, let $E_q \triangleq \Sigma \setminus S$ be the error region. We propagate it backward using the weakest possible precondition cite incorrectness logic / weakest possible preconditions:

$$\mathsf{wpp}(\mathsf{P}, Q) \triangleq \{\sigma \in \Sigma \mid [\![\mathsf{P}]\!]_C(\{\sigma\}) \cap Q \neq \emptyset\}.$$

Thus $\mathsf{wpp}(\mathsf{P}, Q)$ is existential, unlike the universal weakest precondition of Section 5.2. In fact, $\mathsf{wpp}(\mathsf{P}, Q) = \Sigma \setminus \mathsf{wp}(\mathsf{P}, \Sigma \setminus Q)$. Assertions are observational, so $\mathsf{wpp}(\texttt{assert}(S), Q) = Q$. For a summary loop edge, we use $\mathsf{wpp}(\mathsf{loop}(\mathsf{P}_0^*), Q)$ to abbreviate $\mathsf{wpp}(\mathsf{P}_0^*, Q)$.

The error-reproducing input region is $F_q \triangleq I \cap \mathsf{wpp}(\Pi_{\mathsf{entry}\rightsquigarrow q}, E_q)$, which is nonempty for a real alarm. Choose any finite $W_q \subseteq F_q$ and set $R^+ \triangleq R \cup W_q$. For an assertion site r, write $U_R(r, I)$ for $U_R(\Pi_{\mathsf{entry}\rightsquigarrow r}, I)$, equivalently $\delta_{\mathcal{A}}(\alpha(C_r(R)), A_r(I))$.

Corollary D.1 (Soundness of posterior upper-bound refinement). *Let $R \subseteq I$, let $\texttt{assert}_q(S)$ be a validated real error, and let $W_q \subseteq I \cap \mathsf{wpp}(\Pi_{\mathsf{entry}\rightsquigarrow q}, \Sigma \setminus S)$. For $R^+ = R \cup W_q$, every assertion site r satisfies*

$$\mathsf{err}_{\mathcal{A}}(\Pi_{\mathsf{entry}\rightsquigarrow r}, I) \leq U_{R^+}(r, I) \leq U_R(r, I).$$

Proof. Since $W_q \subseteq I$, we have $R \subseteq R^+ \subseteq I$. The result follows by applying Theorem 6.1 to $\Pi_{\mathsf{entry}\rightsquigarrow r}$. □

The condition $W_q \subseteq F_q$ directs the new budget toward behavior exposed by feedback. Soundness needs only $R^+ \subseteq I$, so any genuine added input cannot worsen the bound. Prior guarantees are also preserved. A generator remains a generator, and an h-cover becomes an h^+-cover for some $h^+ \leq h$. Exact wpp computation is semantic and may be approximated by a counterexample trace, bounded unfolding, symbolic execution, or backward analysis with concrete validation of candidates. No loop-internal representatives are retained, because the added states are always inputs in I.

Example. In the two-dimensional polyhedral domain with area distance, let $I = \{0, 1, \ldots, 1000\}^2$ and $R = \{(10i, 10j) \mid 0 \leq i, j \leq 100\}$. This grid has $|R| = 101^2$ versus $|I| = 1001^2$. It preserves $\alpha_{\mathsf{Poly}}(I) = [0, 1000] \times [0, 1000]$ exactly and forms a $5\sqrt{2}$-cover of I. Thus, before the relevant restriction is known, it is already a fine globally distributed representative set.

Now consider the path $x := x+3; (x \geq 990)?; \texttt{assert}_q(x > 990)$. Both $\alpha_{\mathsf{Poly}}(C_q(I))$ and $A_q(I)$ are $[990, 1003] \times [0, 1000]$, so the exact imprecision is zero. The grid nevertheless aligns poorly with this

guard. Its representatives only contribute input layers $x = 990$ and $x = 1000$, reaching $x = 993$ and $x = 1003$. Consequently, $\alpha_{\mathsf{Poly}}(C_q(R)) = [993, 1003] \times [0, 1000]$ and $U_R(q, I) = 13000 - 10000 = 3000$.

The confirmed error then identifies the missing boundary layer. Backward propagation gives $F_q = \{987\} \times \{0, 1, \ldots, 1000\}$ in input space. Choosing $W_q = \{(987, 0), (987, 1000)\}$ adds the missing edge at $x = 990$. Together with the original representative layers, this yields $\alpha_{\mathsf{Poly}}(C_q(R^+)) = A_q(I)$. Therefore the upper bound drops from 3000 to 0.

Another posterior source can be the collection of restrictions encountered along a relevant path program, such as guards. These restrictions can also be propagated backward by weakest preconditions. Rather than sampling only to minimize a global covering radius, a posterior-guided strategy can place additional representatives near the boundaries of the resulting precondition regions. The intuition is that guards remove or separate program states, and precision is often lost near the induced boundaries. Representatives near those boundaries are more likely to preserve the local geometric information needed for a tight upper-bound estimate.